

|               |                            |              |          |               |            |
|---------------|----------------------------|--------------|----------|---------------|------------|
| DEPARTAMENTO: | Ciencias de la Computación | CARRERA:     | Software |               |            |
| ASIGNATURA:   | Pruebas de Software        | NIVEL:       | Sexto    | FECHA:        | 10/07/2025 |
| DOCENTE:      | Ing. Luis Castillo         | PRÁCTICA N°: | 2        | CALIFICACIÓN: |            |

## Pruebas de sistema aplicadas sobre API REST

Carlos Sebastián Rivera Espín

### RESUMEN

En la presente práctica de laboratorio se ejecutaron pruebas de sistema sobre una API REST desarrollada para la gestión de reservas en una biblioteca. El objetivo fue evaluar su comportamiento en situaciones funcionales normales, de estrés y de seguridad. Se utilizó Postman para validar los endpoints, JMeter para realizar pruebas de carga con hasta 500 usuarios simulados y OWASP ZAP para identificar posibles vulnerabilidades. Se observaron resultados positivos en la integridad y estabilidad del sistema bajo condiciones controladas. Las pruebas revelaron algunos puntos de mejora, especialmente en cuanto a validación de entradas y protección ante accesos indebidos. El ejercicio permitió reforzar conocimientos sobre pruebas automatizadas y análisis de comportamiento en entornos reales de producción.

**Palabras Claves:** Pruebas de sistema, API REST, Seguridad web

### 1. INTRODUCCIÓN:

En el desarrollo de sistemas web, las pruebas de software desempeñan un papel fundamental para garantizar la calidad, seguridad y correcto funcionamiento de las aplicaciones. Esta práctica de laboratorio tuvo como finalidad aplicar diferentes tipos de pruebas sobre el backend de una API REST desarrollada para la gestión de reservas bibliotecarias.

La actividad se estructuró en cuatro fases: pruebas funcionales y de regresión utilizando Postman; pruebas de carga mediante Apache JMeter; y pruebas de seguridad con OWASP ZAP. Cada una de estas herramientas permitió analizar distintos aspectos del sistema, desde su comportamiento bajo uso normal hasta su respuesta ante ataques o condiciones extremas de carga.

El trabajo fue desarrollado en un entorno controlado de laboratorio, promoviendo el manejo disciplinado de los recursos, la correcta configuración del ambiente de desarrollo, y la aplicación metódica de técnicas de prueba. Gracias a este enfoque, fue posible detectar errores, validar restricciones y comprobar la estabilidad de la aplicación frente a escenarios reales y adversos.

### 2. OBJETIVO(S):

#### 2.1 Objetivo general:

Aplicar de manera sistemática pruebas funcionales, de carga y de seguridad sobre una API RESTful para la gestión de reservas, utilizando herramientas especializadas con el fin de validar su comportamiento, estabilidad y resistencia ante diferentes condiciones operativas en un entorno de laboratorio controlado.

#### 2.2 Objetivos específicos:

- Evaluar el correcto funcionamiento de los endpoints de la API mediante pruebas manuales en Postman.
- Verificar que los cambios implementados en el código no afecten funcionalidades existentes, aplicando pruebas de regresión.
- Simular múltiples usuarios concurrentes para medir el rendimiento del sistema mediante Apache JMeter.
- Identificar posibles vulnerabilidades de seguridad y validar el control de accesos utilizando OWASP ZAP.
- Documentar los resultados obtenidos a través de capturas, análisis de errores, tiempos de respuesta y alertas de seguridad.
- Promover buenas prácticas de pruebas en el ciclo de vida del desarrollo de software.

### 3. MARCO TEÓRICO:

Durante esta práctica se utilizaron tres herramientas ampliamente reconocidas en el desarrollo y aseguramiento de la calidad de sistemas web: Postman, Apache JMeter y OWASP ZAP. Cada una cumple una función específica dentro del ciclo de pruebas.

- **Postman:** Postman es una plataforma que permite construir, probar y documentar APIs REST. Facilita la ejecución de pruebas funcionales mediante la creación de colecciones de solicitudes HTTP (GET, POST, PUT, DELETE), validando los datos de respuesta, encabezados, estados y tokens de autenticación.
- **Apache JMeter:** JMeter es una herramienta de código abierto diseñada para realizar pruebas de rendimiento y carga. Simula múltiples usuarios concurrentes que envían solicitudes al servidor, lo que permite analizar tiempos de respuesta, tasas de error, cuellos de botella y el comportamiento del sistema ante saturación.
- **OWASP ZAP:** El OWASP Zed Attack Proxy es una herramienta de análisis de seguridad mantenida por la comunidad OWASP. Su propósito es encontrar vulnerabilidades comunes en aplicaciones web, como inyecciones, exposiciones de rutas, validaciones débiles y malas prácticas de autenticación. Implementa funciones como escaneo pasivo, spidering, escaneo activo y fuzzing.
- **Pruebas de regresión:** Estas pruebas se ejecutan después de realizar cambios en el sistema para asegurar que funcionalidades previamente estables no hayan sido afectadas. Son esenciales en entornos ágiles donde el software evoluciona constantemente.
- **Pruebas de carga:** Se utilizan para medir el rendimiento del sistema bajo diferentes volúmenes de usuarios concurrentes. Permiten identificar los límites del sistema, su tiempo de respuesta y posibles fallos bajo condiciones extremas.
- **Fuzzing:** Es una técnica de prueba en la que se envían entradas aleatorias o maliciosas a los campos de un formulario o petición API, con el fin de detectar vulnerabilidades o comportamientos inesperados, como caídas del servidor o respuestas con información sensible.

### 4. DESCRIPCIÓN DEL PROCEDIMIENTO:

A continuación, se detalla el desarrollo completo de la práctica de laboratorio, la cual fue organizada en cuatro partes principales. Cada sección describe los pasos realizados, las herramientas empleadas y los objetivos específicos cumplidos en cada fase. Desde la configuración del entorno de ejecución hasta la aplicación de pruebas automatizadas de carga y seguridad, se aplicaron buenas prácticas de pruebas de software con el propósito de validar la funcionalidad, el rendimiento y la robustez del sistema. Las evidencias, observaciones y resultados obtenidos se documentan a continuación de manera ordenada.

#### 4.1 Materiales

- **Node.js:** Plataforma para ejecutar el backend de la API REST desarrollada en JavaScript.
- **MongoDB y MongoDB Compass:** Base de datos NoSQL para almacenamiento de usuarios y reservas.
- **Postman:** Herramienta para ejecutar pruebas funcionales y de regresión sobre la API.
- **Apache JMeter:** Herramienta de pruebas de carga y estrés, utilizada para simular usuarios concurrentes.
- **OWASP ZAP (Zed Attack Proxy):** Herramienta de seguridad para pruebas de spidering, escaneo activo y fuzzing.
- **Visual Studio Code:** Editor de código utilizado para explorar y modificar el backend.
- **Navegador Google Chrome:** Utilizado para monitoreo del entorno y visualización de ZAP.
- **Docker Desktop** (opcional): Para levantar contenedores en caso de pruebas aisladas o futuras implementaciones automatizadas.

#### 4.2 Muestras

Para realizar las pruebas se utilizó el backend entregado por el docente, el cual forma parte de una aplicación web para la gestión de reservas en una biblioteca universitaria. Las pruebas se realizaron sobre los siguientes módulos de la API:

- Autenticación de usuarios (/api/auth)
- Registro, inicio de sesión, validación de token.
- Gestión de reservas (/api/reservas)

- Crear reserva, obtener reservas, cancelar reserva.
- Validaciones y restricciones
- Reglas de negocio como horarios válidos y restricción de reservas los domingos.

### 4.3 Métodos

#### Parte 1 - Establecer el ambiente de desarrollo

En esta etapa inicial se prepara el entorno de ejecución del backend de la aplicación de reservas. Esta configuración es esencial para que el sistema pueda ser probado correctamente en las siguientes fases.

#### Paso 1: Descargar el backend proporcionado por el docente

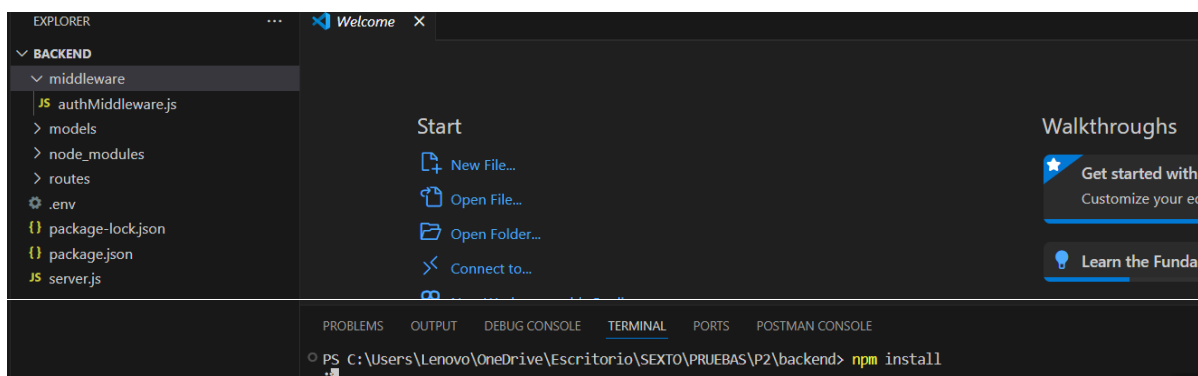


**Fig 1. Backend descargado**

#### Paso 2: Ejecutar el backend entregado por el docente

##### 1. Instalación de dependencias:

- Se utiliza el comando npm install en la carpeta del proyecto para instalar todos los paquetes necesarios definidos en el archivo package.json.

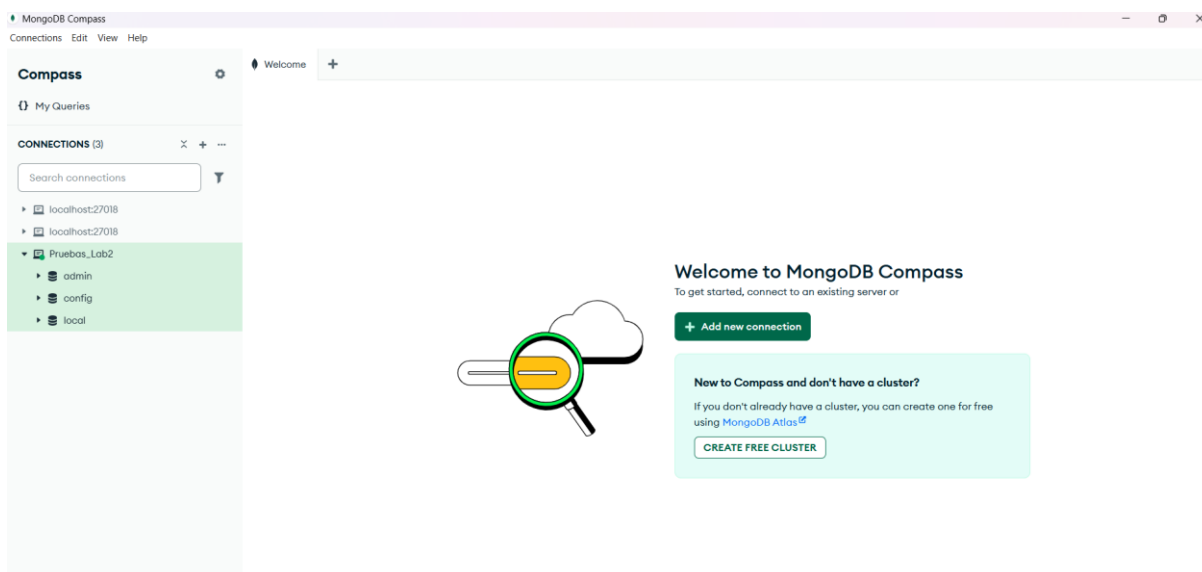


**Fig 2. Instalación de dependencias**

Pantalla de la terminal ejecutando npm install y mostrando los paquetes instalados correctamente.

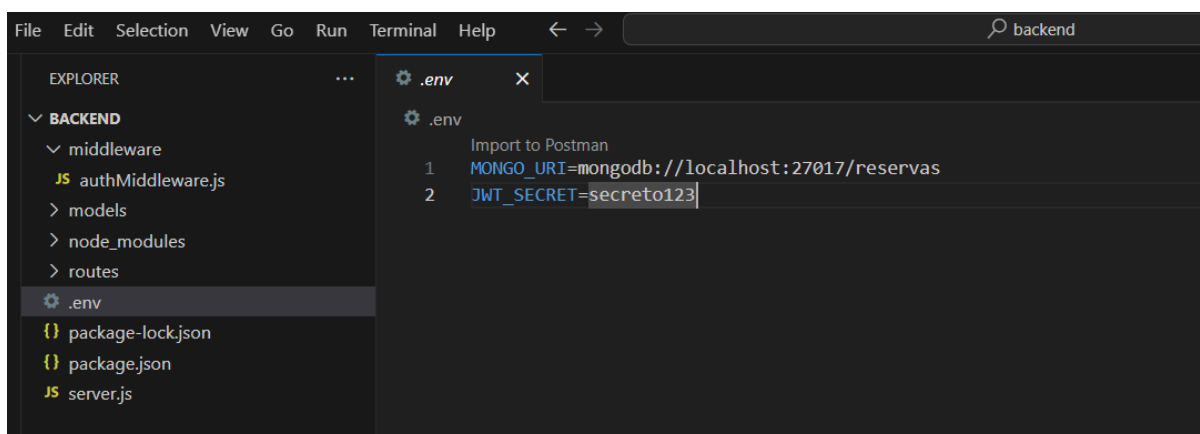
##### 2. Conexión con MongoDB:

- Se verifica la conexión con la base de datos mediante el archivo .env, donde se encuentra la IP o URI de conexión. Esta configuración asegura que el backend pueda comunicarse con MongoDB.



**Fig 3. Conexión con Mongo – archivo “.env”**

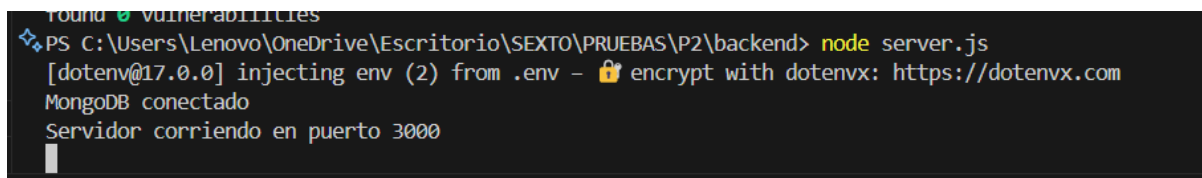
- Fragmento del archivo .env donde se muestre la URI de conexión a MongoDB.



**Fig 4. Archivo “.env”**

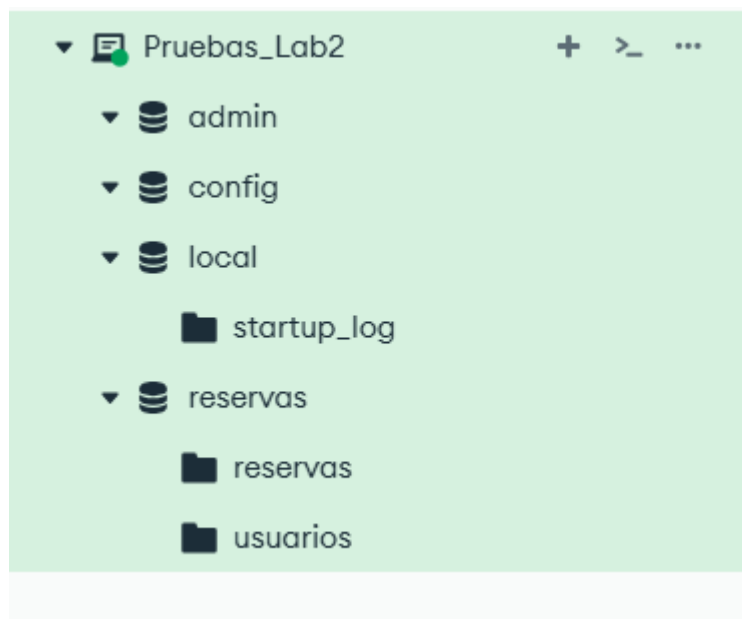
### 3. Ejecución del servidor:

- Se inicia el servidor backend con el comando node server.js.



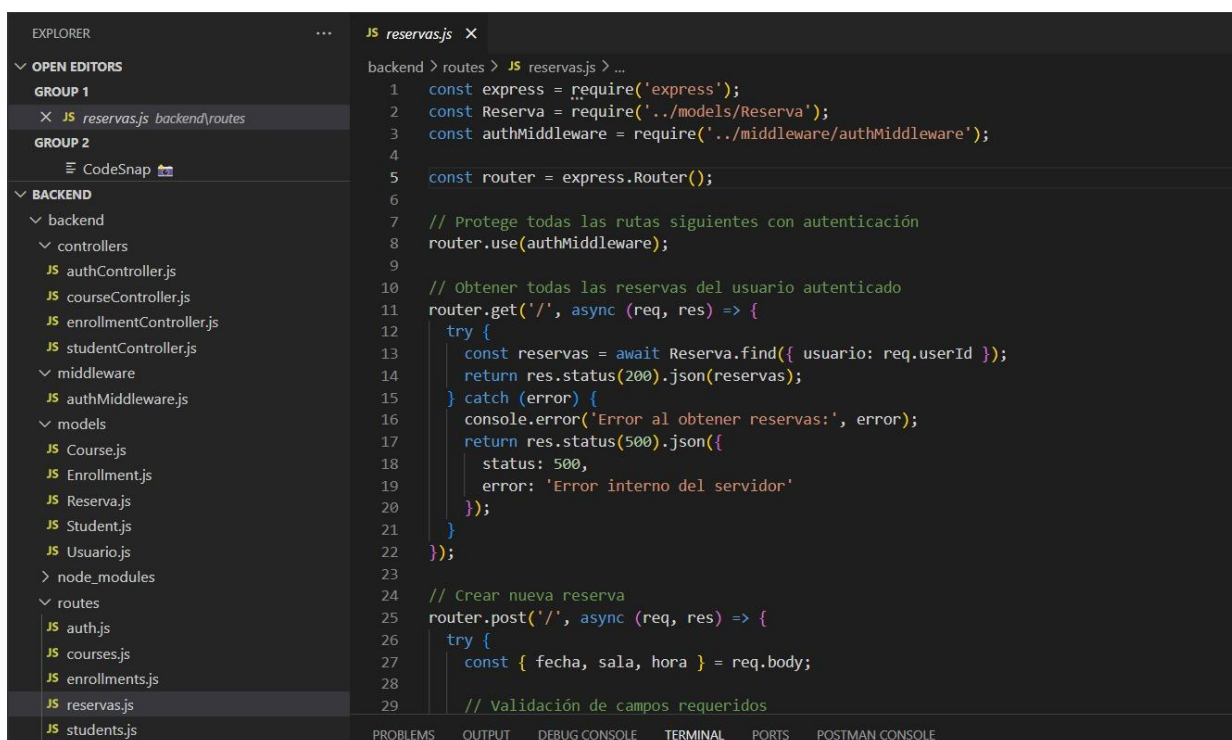
**Fig 5. Ejecución del comando “node server.js”**

- Consola mostrando el mensaje “Servidor corriendo en puerto 3000” o similar.
- ### 4. Verificación de base de datos:
- Se accede a MongoDB en este caso Mongo Compass para validar que las colecciones necesarias (usuarios y reservas) se hayan creado.



**Fig 6. Colecciones agregadas en MongoCompas**

- Interfaz de MongoDB Compass mostrando las colecciones creadas.
- 5. **Revisión de código backend:**
  - Se exploran los archivos de la API para entender las rutas y funcionalidades implementadas (por ejemplo: routes/reservas.js, routes/auth.js, etc.).



**Fig 7. Rutas del backend – reservas.js y auth.js**

- Visualización de uno de los archivos del backend mostrando el manejo de rutas.

## SECCIÓN: Parte 2 - Realizar pruebas con POSTMAN

En esta fase se realizan pruebas funcionales sobre los endpoints del backend mediante la herramienta Postman. Se verifica el correcto funcionamiento de las operaciones de autenticación, creación, obtención y cancelación de reservas. Además, se ejecutan pruebas de regresión y seguridad básica para evaluar la robustez de la aplicación.

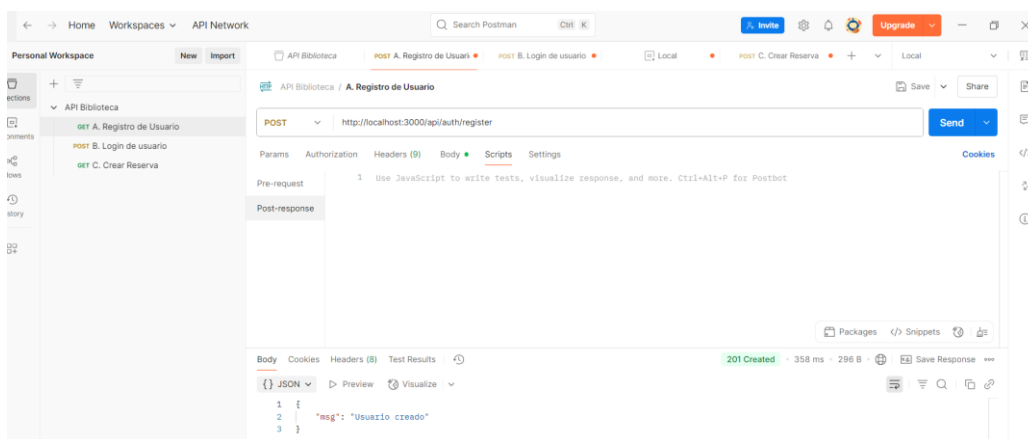
### Paso 1: Preparación de Colección en Postman

- Se creó una nueva colección llamada **API Biblioteca**.



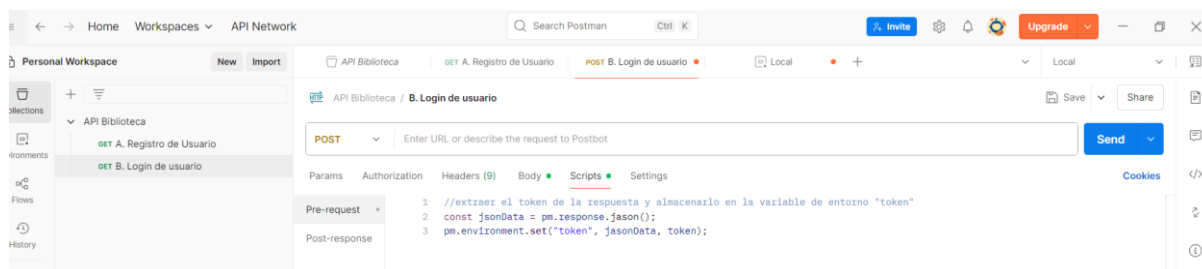
**Fig 8. Creación de la colección Api Biblioteca en POSTMAN**

- Se añadieron los siguientes endpoints (request):
  - Registro de usuario (POST /api/auth/register)



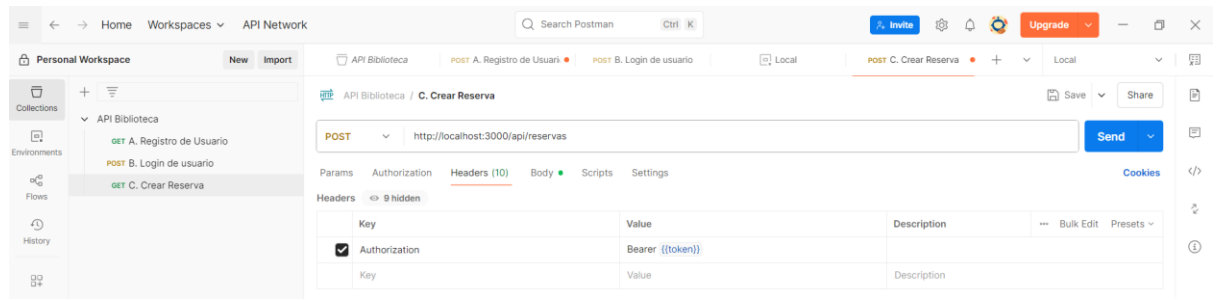
**Fig 9. Creación de un nuevo Usuario - POSTMAN**

- Login de usuario (POST /api/auth/login)



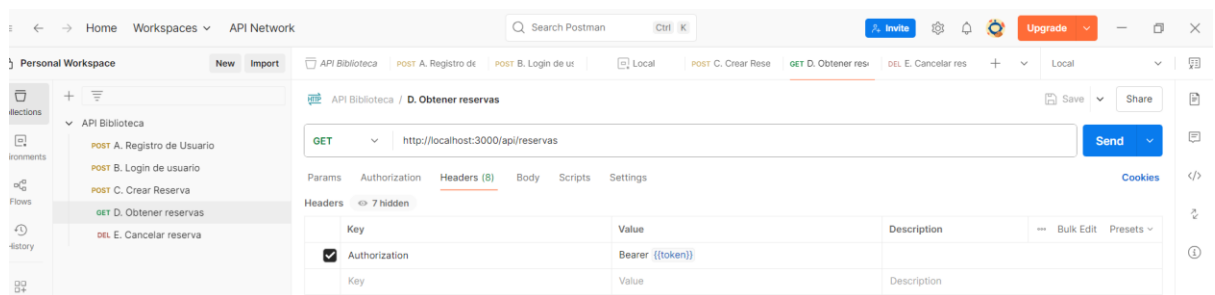
**Fig 10. Configuración de Login - Token**

- Crear reserva (POST /api/reservas)



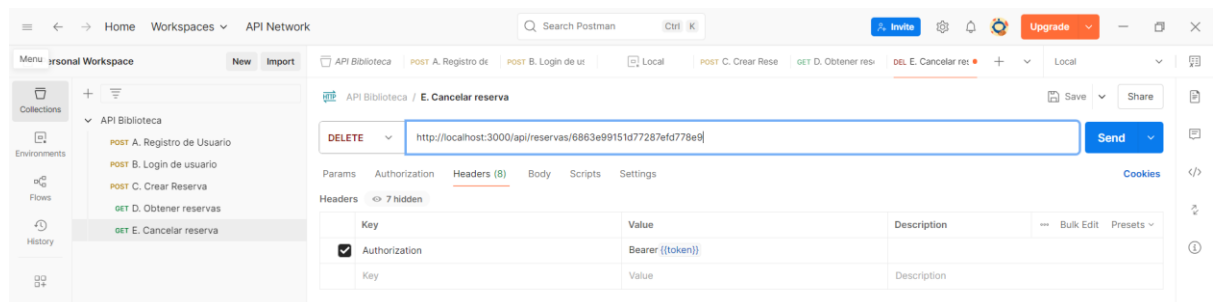
**Fig 11. Configuración de Reserva - POSTMAN**

- Obtener reservas (GET /api/reservas)



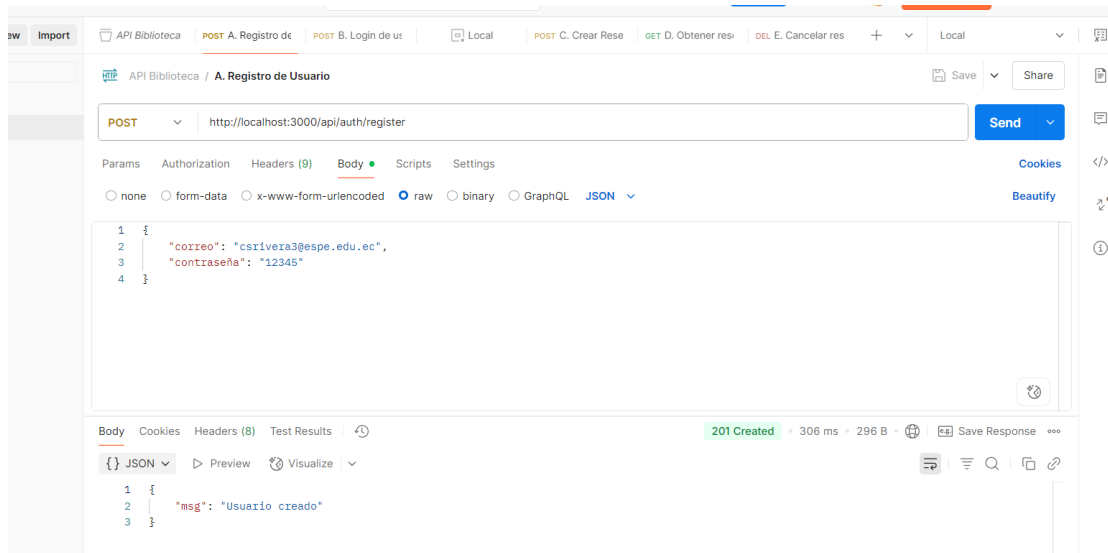
**Fig 11. Configuración de Obtener Reserva - POSTMAN**

- Cancelar reserva (DELETE /api/reservas/:id)



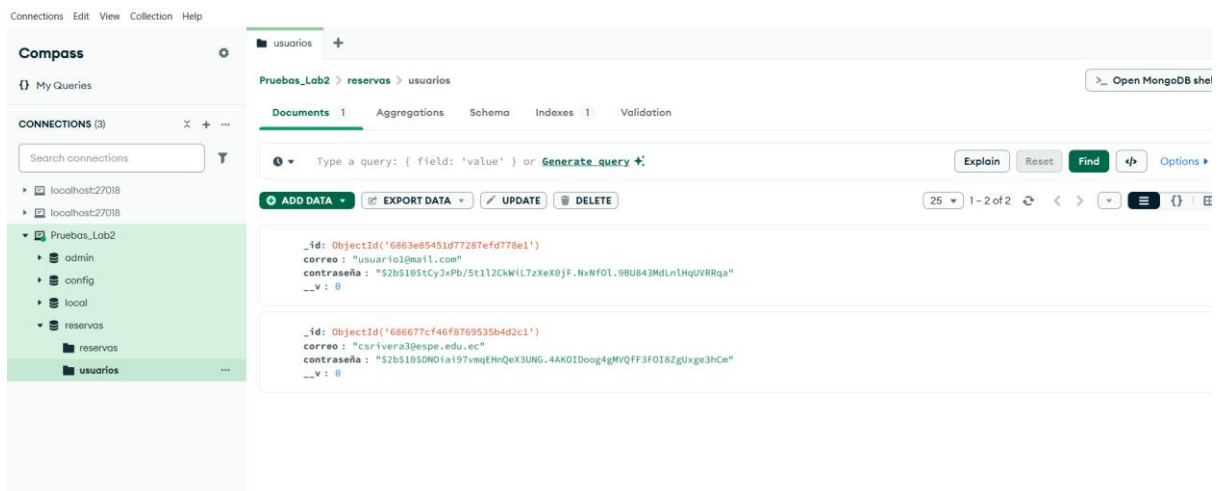
**Fig 14. Configuración de Eliminar Reserva - POSTMAN**

- Petición de registro con respuesta exitosa.



**Fig 15. Creación de Usuario exitosa**

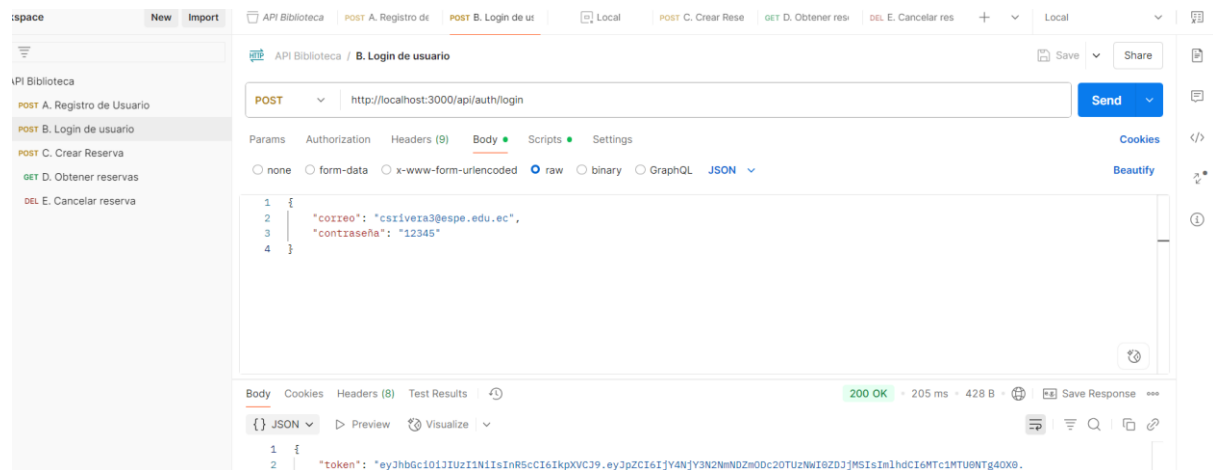
- Usuario Guardado en Mongo



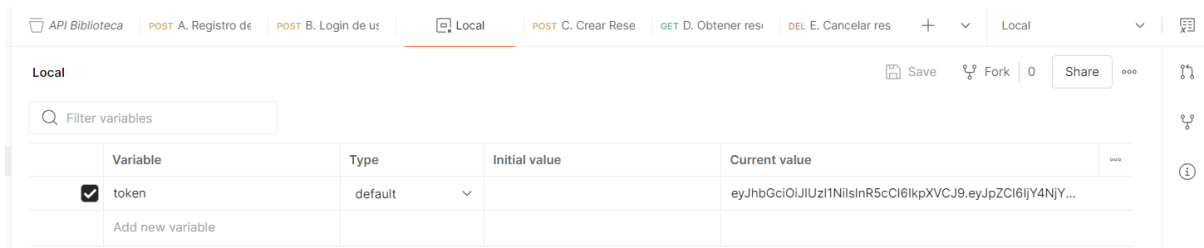
**Fig 16. Carga de nuevo usuario - MongoCompass**

- Petición de login mostrando el token recibido.



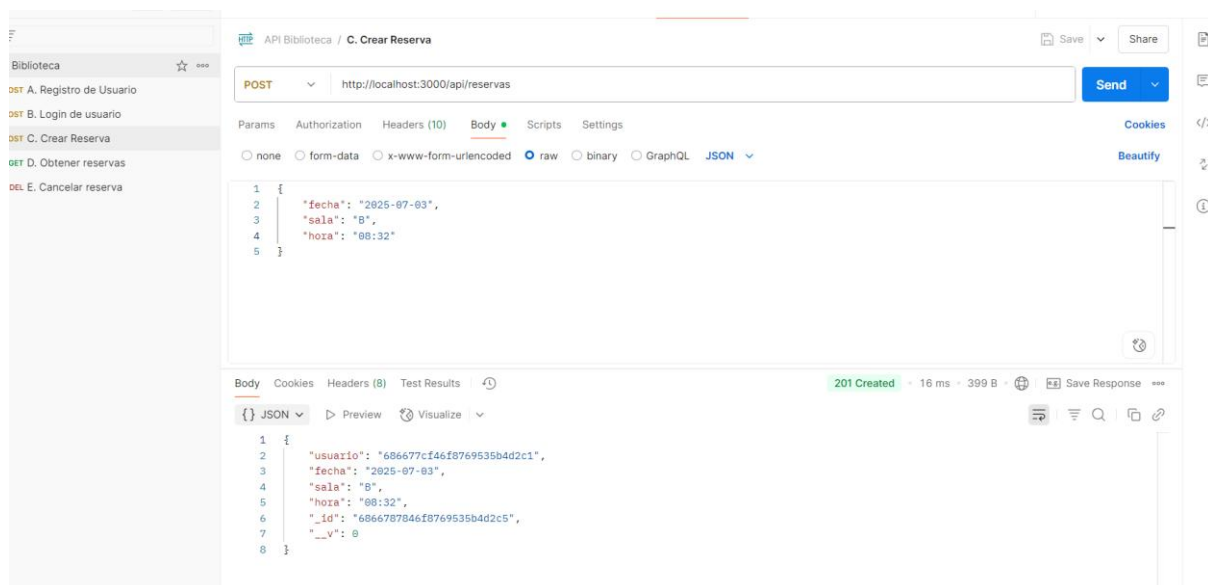


**Fig 16. Login de Usuario Creado - POSTMAN**



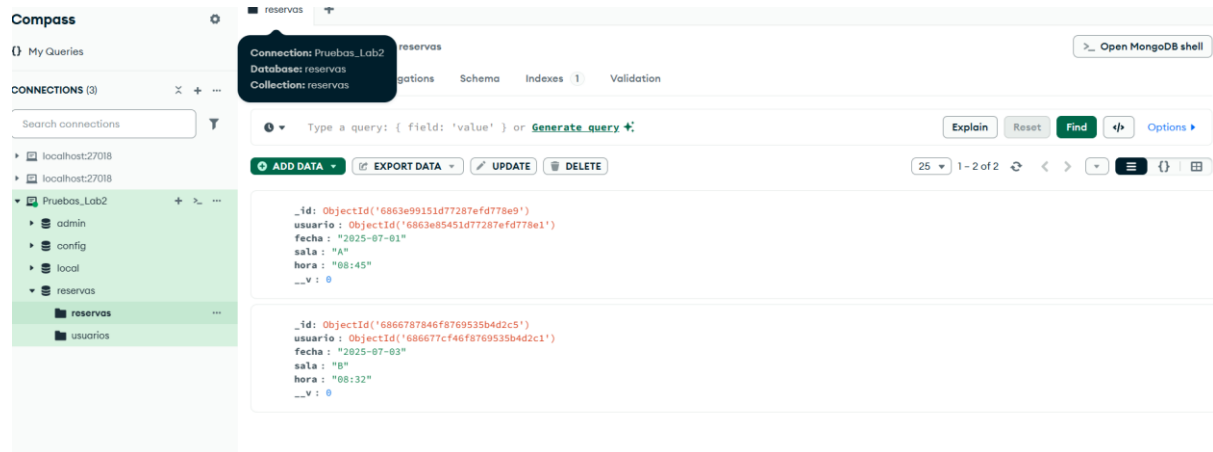
**Fig 17. Generación de token - Local**

- Petición de creación de reserva.



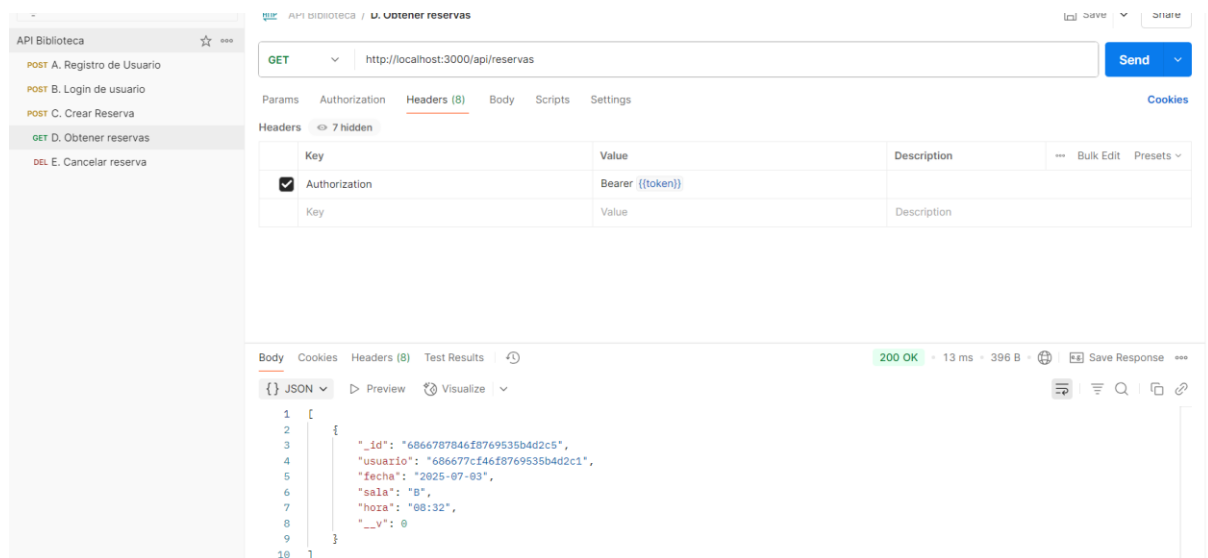
**Fig 18. Creación de nueva Reserva - POSTMAN**

- Reserva creada y guardada en mongo



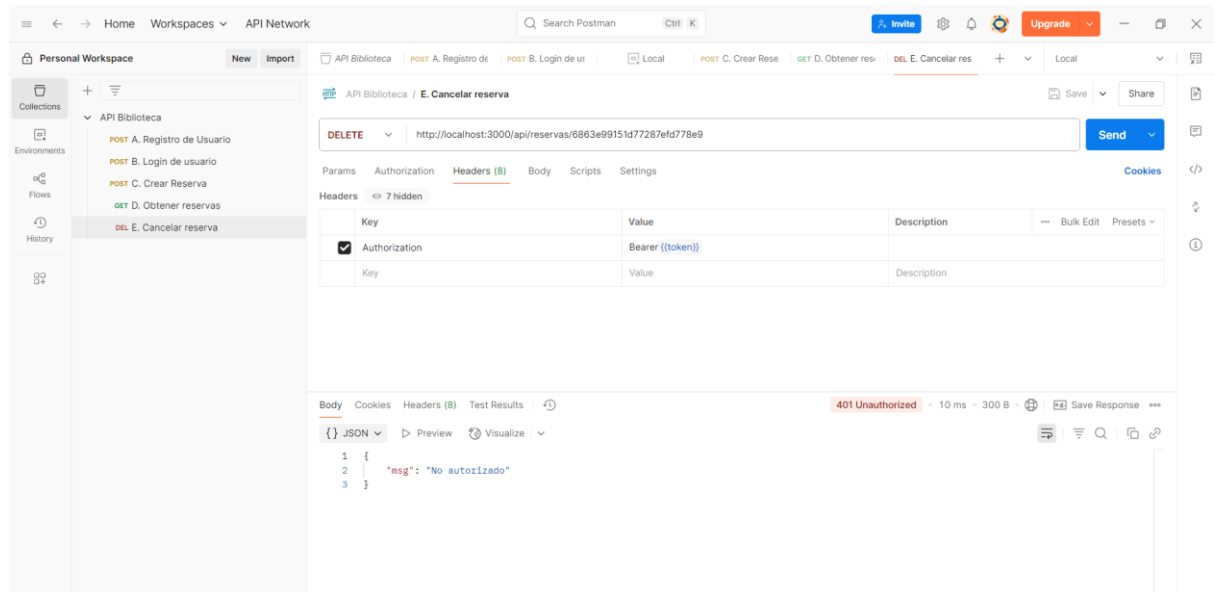
**Fig 18. Carga de nueva Reserva - MongoCompass**

- Petición de obtención de reservas.



**Fig 19. Obtención de la reserva creada - POSTMAN**

- Cancelación de reserva.

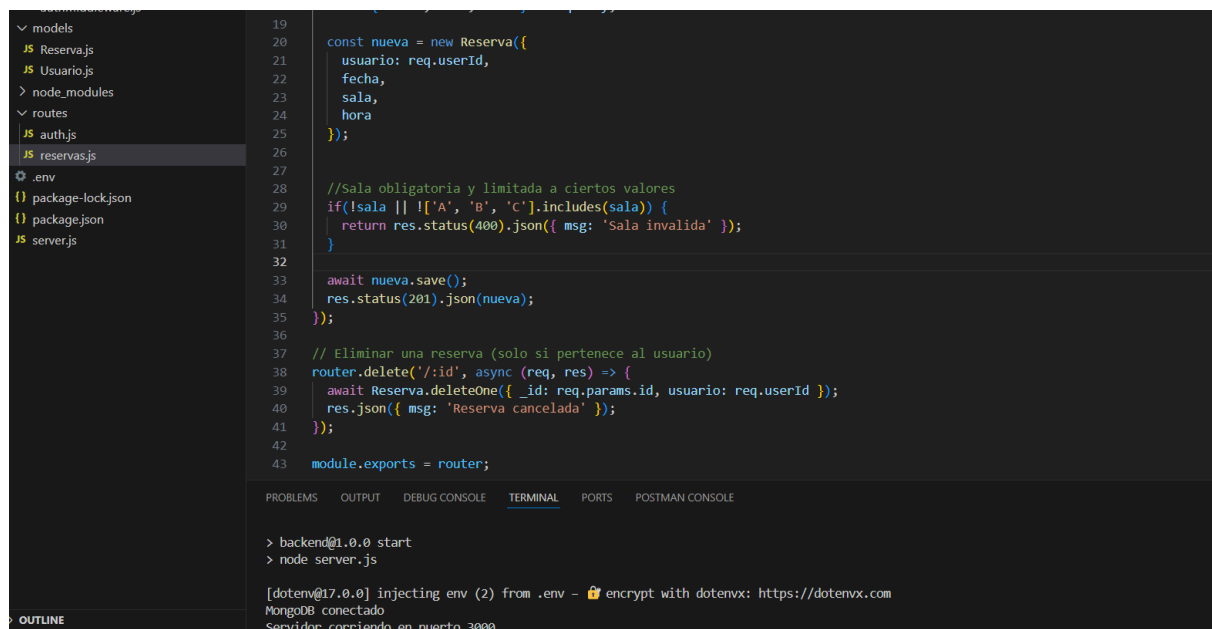


**Fig 20. Eliminación de reserva creada - POSTMAN**

Sale error, pero vamos a mejorar el código del backend y además validar ciertos parámetros.

## Paso 2: Prueba de regresión

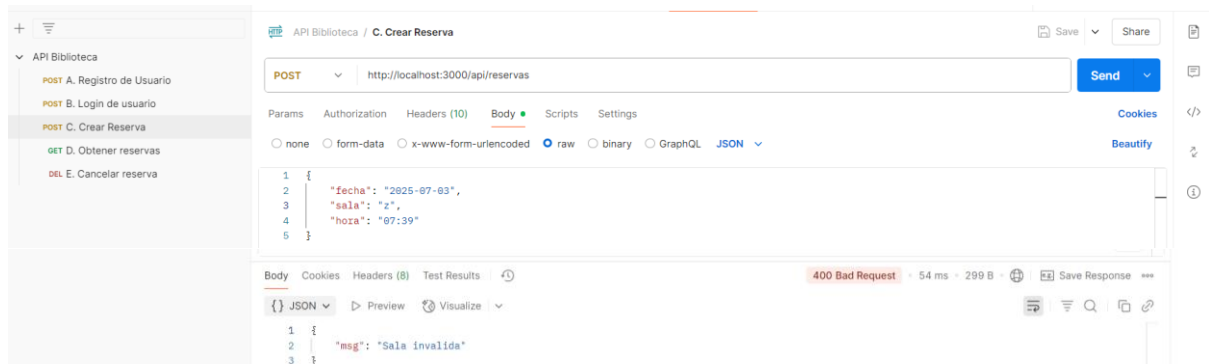
1. Se ejecutaron todos los endpoints usando un conjunto base de datos válidos.
2. Se aplicó un cambio en la API: el campo sala ahora solo acepta los valores 'A', 'B' o 'C'. Cualquier otro valor devuelve error 400 Sala inválida.



**Fig 21. Mejora de código – Backend (reservas.js)**

3. Se volvió a ejecutar la creación de reserva con valores válidos e inválidos para el campo sala.

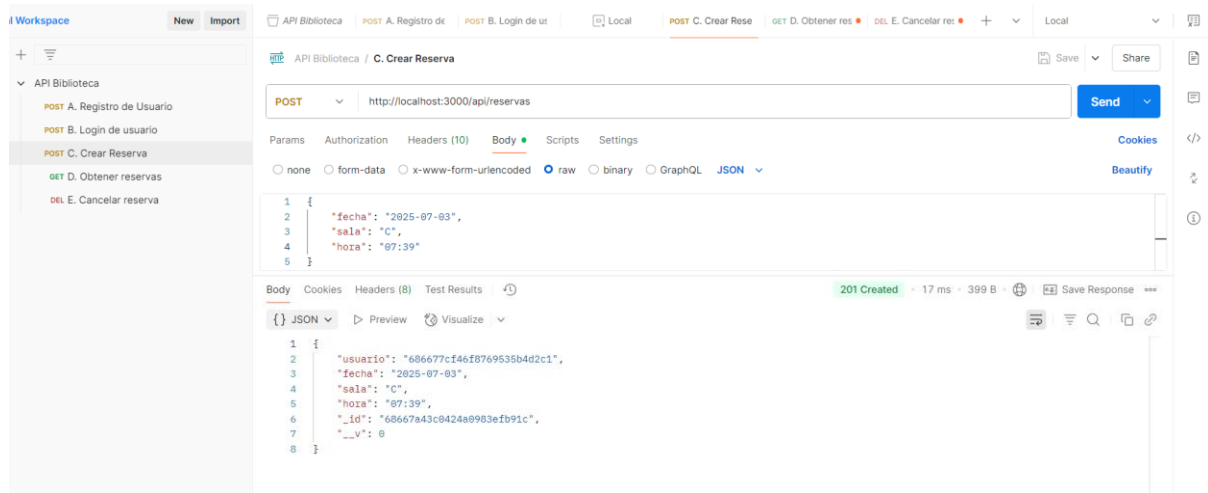
- Prueba con un parámetro “SALA Z” invalido



**Fig 22. Prueba con parámetro “SALA” invalido**

Sale invalida porque **Z** esta fuera de los parámetros

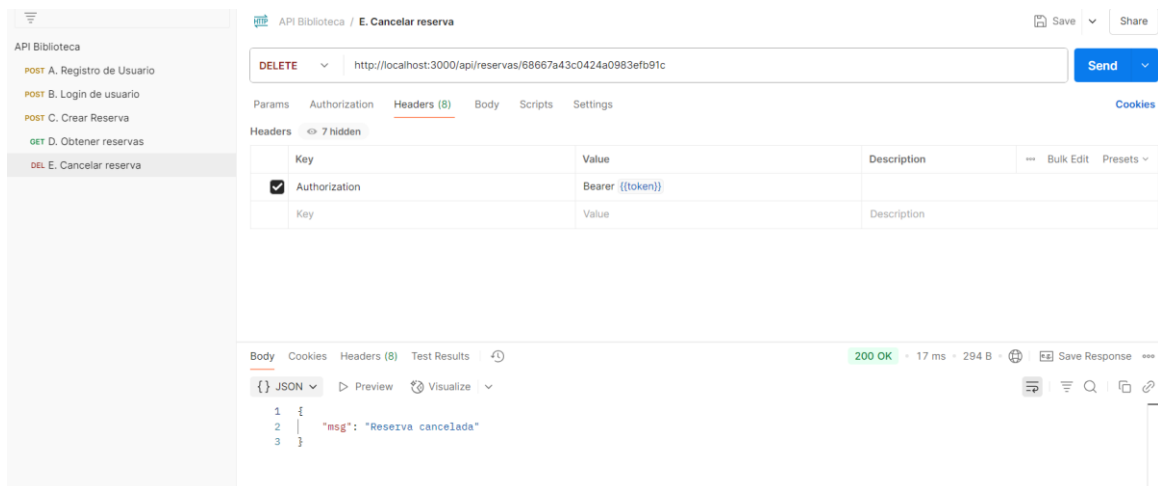
- Prueba con “SALA C” valido



**Fig 22. Prueba con parámetro “SALA” valido**

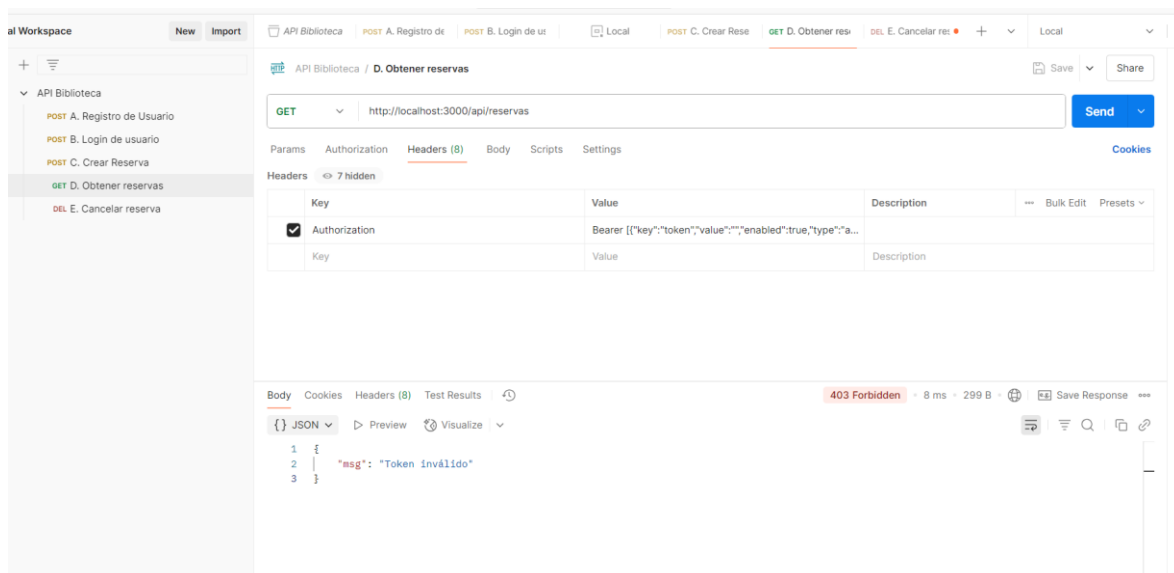
Sala creada porque está dentro de los parámetros

- Prueba de cancelación de reserva con parámetros de sala



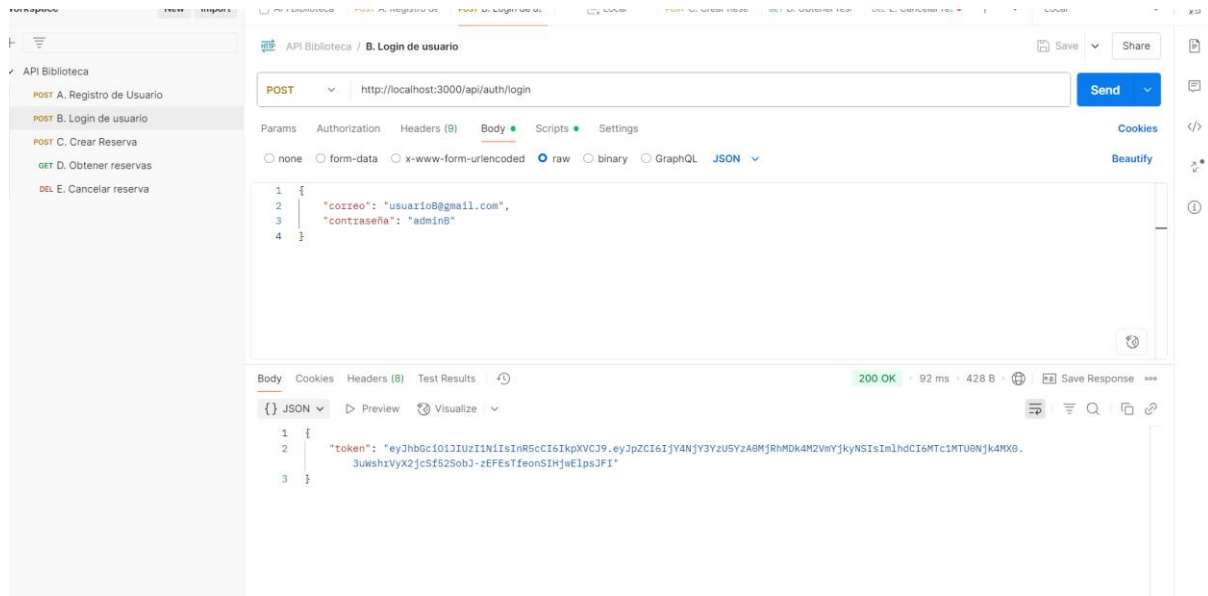
**Fig 23. Prueba de cancelación de reserva – Parametro modificado (exitosa)**

- Prueba de seguridad básica TOKEN



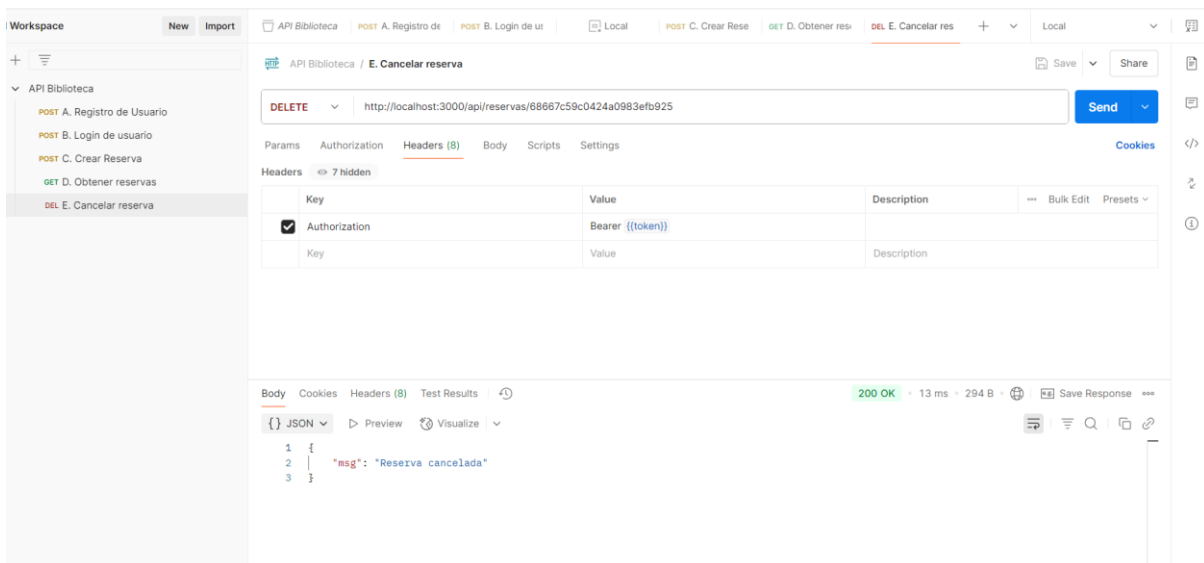
**Fig 23. Prueba de Token invalido**

- Prueba de eliminar reserva con un diferente usuario



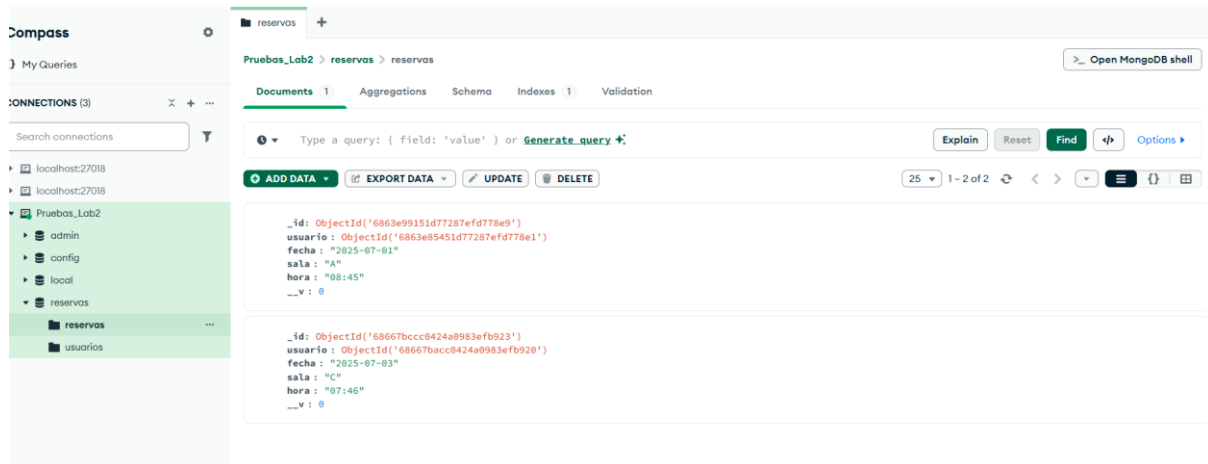
**Fig 24. Prueba de Token invalido**

- Se colocó el id del usuario “b” y en el **DELETE** para borrar la reserva del del usuario “a”



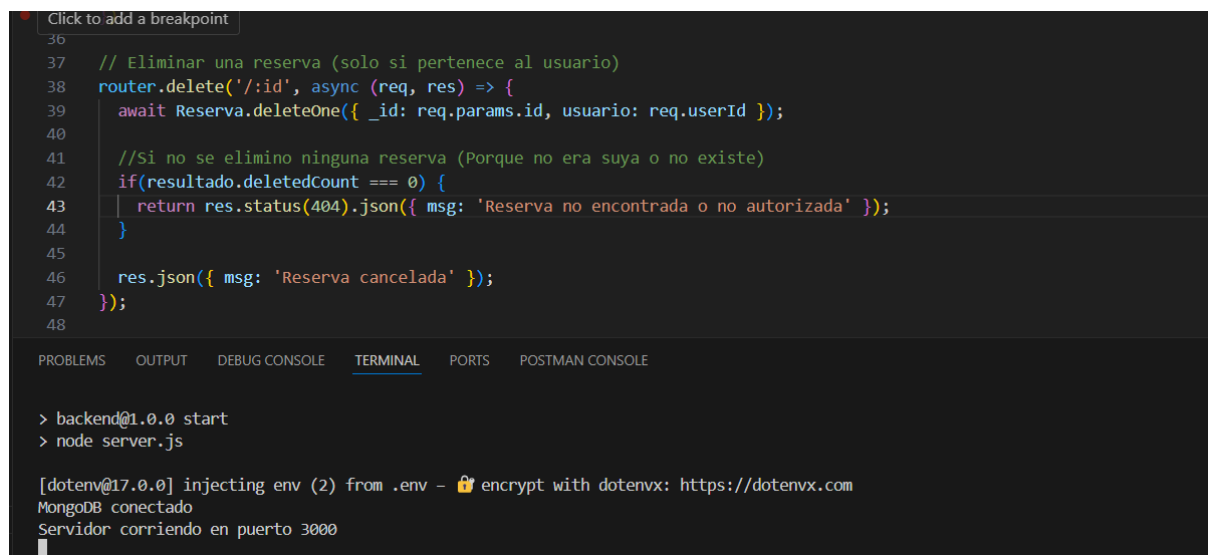
**Fig 25. Prueba de cancelación de reserva – Id usuario invalido**

El mensaje nos salía “Reserva Cancelada” pero únicamente es el mensaje ya que comprobando en el mongo la reserva no se canceló eso se modifica en el código del backend

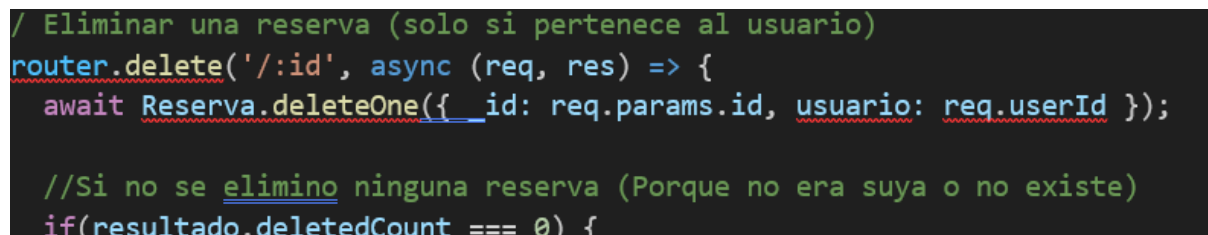


**Fig 26. Verificación de base de datos – Eliminación reserva ()**

- Modificación de código



**Fig 27. Modificación de Backend – reservas.js**



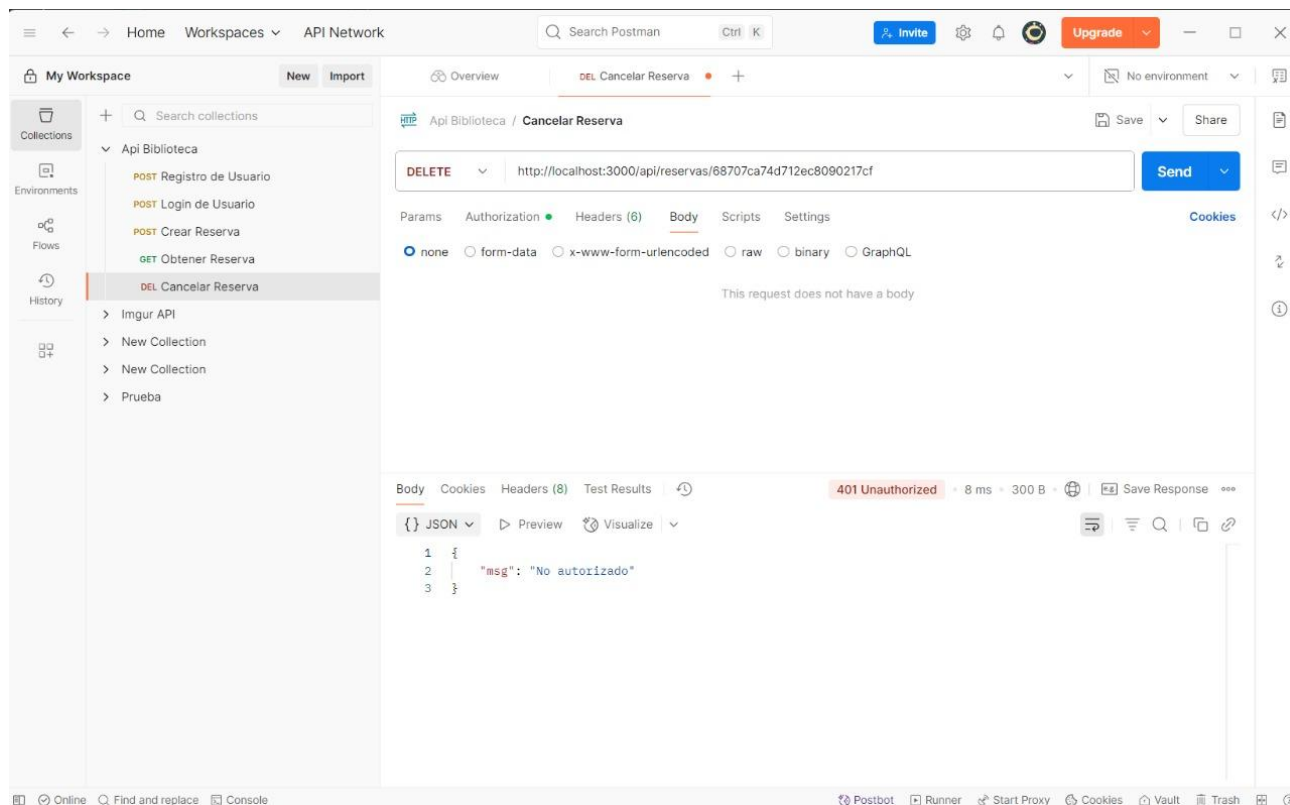
**Fig 28. Código agregado – Parámetro “A-B-C”**

- Acceso sin token

Se intentó acceder a /api/reservas sin enviar el token de autenticación. El sistema devolvió un error 401 Unauthorized.

- Token inválido

Se usó un token mal formado o manipulado y se obtuvo un error 401.



**Fig 29. Ingreso de token - Invalido**

## SECCIÓN: Parte 3 - Script de Pruebas de Estrés con JMeter

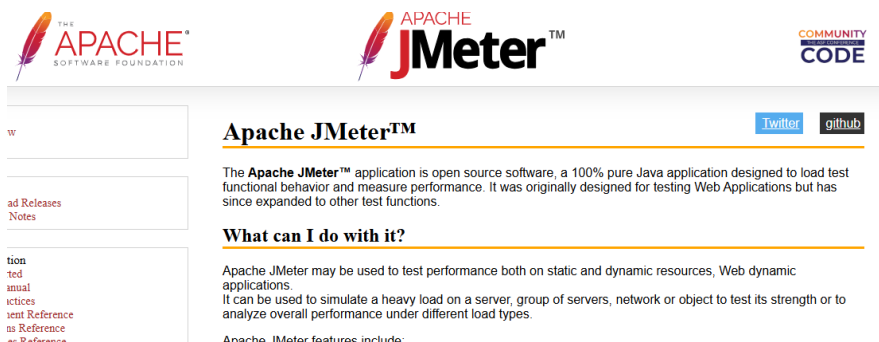
En esta sección se utilizan pruebas de carga (estrés) con Apache JMeter para evaluar el comportamiento del backend bajo un número creciente de usuarios concurrentes. Se simulan múltiples solicitudes de login y creación de reservas para analizar tiempos de respuesta, estabilidad y posibles errores del sistema.

### Paso 1: Descargar y abrir Apache JMeter

Para llevar a cabo las pruebas de carga sobre la API, se utilizó **Apache JMeter**, una herramienta gratuita y de código abierto para pruebas de rendimiento.

#### 1. Descarga de JMeter:

- Se ingresó al sitio oficial: <https://jmeter.apache.org/>



**Fig 30. Ejecución de JMeter**

- Se descargó la versión estable más reciente en formato .zip para Windows.



[FAQ \(Wiki\)](#)
[KEYS](#)
[Tutorials](#)
[Distributed Testing](#)  
[Recording Tests](#)  
[JUnit Sampler](#)  
[Access Log Sampler](#)  
[Extending JMeter](#)
[Community](#)
[Issue Tracking](#)  
[Security](#)  
[Mailing Lists](#)  
[Source Repositories](#)  
[Building and Contributing](#)  
[Project info at Apache](#)  
[Contributors](#)
[Foundation](#)
[The Apache Software Foundation \(ASF\)](#)

## Apache JMeter 5.6.3 (Requires Java 8+)

### Binaries

[apache-jmeter-5.6.3.tgz sha512 pgp](#)  
[apache-jmeter-5.6.3.zip sha512 pgp](#)

### Source

[apache-jmeter-5.6.3\\_src.tgz sha512 pgp](#)  
[apache-jmeter-5.6.3\\_src.zip sha512 pgp](#)

**Fig 31. Descarga del zip de JMeter**

- Se descomprimió el archivo en una carpeta local del equipo.

▼ Hoy

 apache-jmeter-5.6.3

▼ Ayer

**Fig 32. Zip descargado en la maquina**

## 2. Ejecución:

- Se accedió a la carpeta /bin dentro del directorio de JMeter descargado.

|                |        |
|----------------|--------|
| ▼ more         | iam    |
| ..             |        |
| bin            | 3.329  |
| docs           | 63.830 |
| extras         | 114    |
| lib            | 77.928 |
| licenses       | 1.017  |
| printable_docs | 4.392  |
| LICENSE        | 17     |

**Fig 33. Carpeta bin – archivo jmeter.bat**

- Se ejecutó el archivo jmeter.bat (en sistemas Windows).

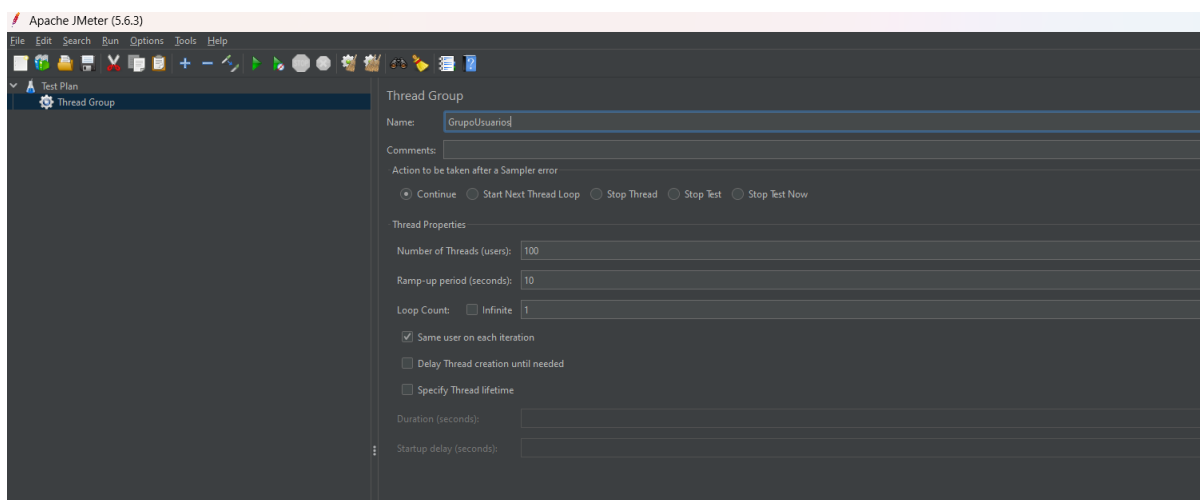
|                   |
|-------------------|
| jaas.conf         |
| jmeter            |
| jmeter.bat        |
| jmeter.properties |
| jmeter.sh         |
| jmeter-n.cmd      |

**Fig 34. Archivo jmeter.bat**

- Esto abrió la interfaz gráfica (GUI) de Apache JMeter, donde se visualizó el árbol de elementos del Plan de Pruebas vacío, listo para configurarse.

## Paso 2: Crear grupo de hilos

1. Clic derecho sobre “Plan de pruebas” → **Agregar** → **Threads (Usuarios)** → **Grupo de Hilos**.
2. Se renombró el grupo a: GrupoUsuarios.
3. Configuración:
  - Número de hilos (usuarios): **100**
  - Período de subida: **1**
  - Recuento de ciclos: **1**

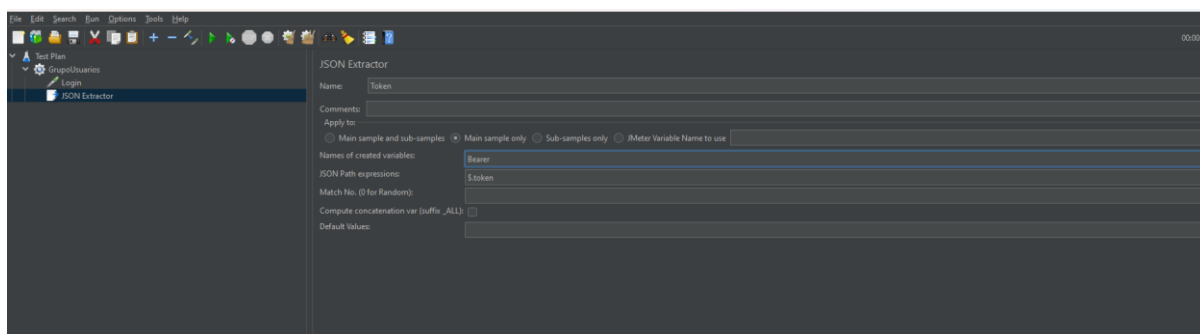


**Fig 35. Creación de grupo de hijos – Grupo Usuarios**

- Grupo de hilos con la configuración para 100 usuarios.

## Paso 3: Crear carpeta de lógica de test (Controlador simple)

1. Clic derecho sobre GrupoUsuarios → **Agregar** → **Lógica de controladores** → **Controlador simple**.
2. Se renombró como: Login.

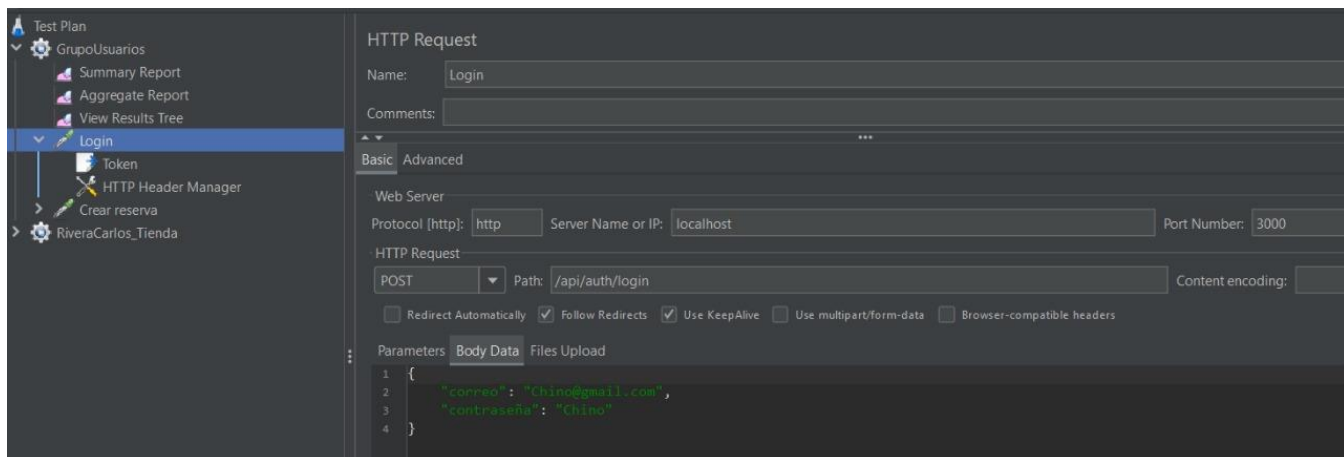


**Fig 36. Creación de Login – Grupo Usuarios**

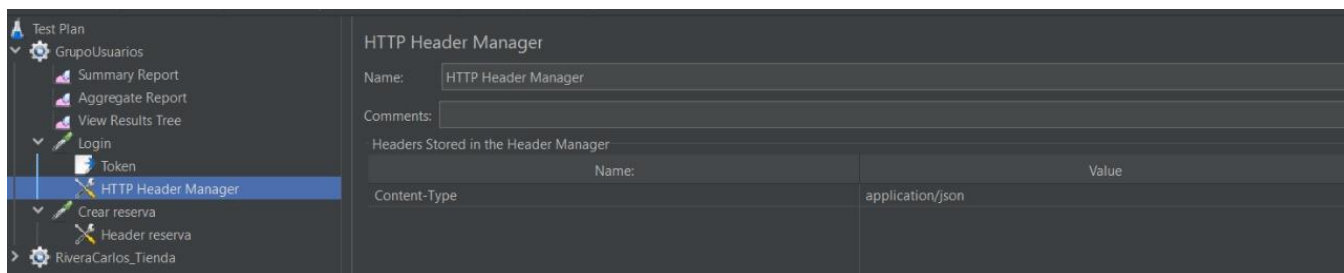
## Paso 4: Crear Peticiones HTTP

1. **Login:**
  - Clic derecho sobre LoginReservaTest → **Agregar** → **Sampler** → **Petición HTTP**
  - Nombre: Login

- Servidor: localhost
- Puerto: 3000
- Método: POST
- Ruta: /api/auth/login
- Parámetros:
  - email: Chino@gmail.com
  - password: Chino



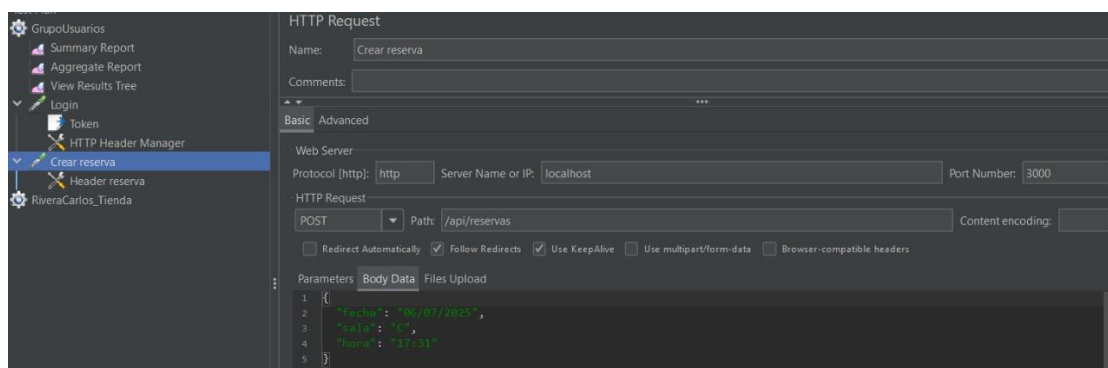
**Fig 37. Configuración de Login – Ruta**



**Fig 38. Configuración de Login – Grupo Usuarios**

## 2. Crear Reserva:

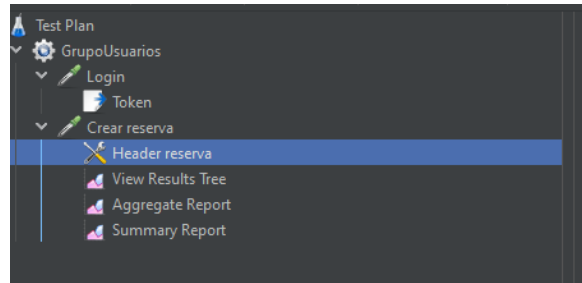
- Se repite el proceso para la ruta /api/reservas usando el método POST con parámetros válidos (ej. sala, fecha, hora).



**Fig 39. Configuración de Crear Reserva – Ruta**

## Paso 5: Agregar Listeners (Reportes)

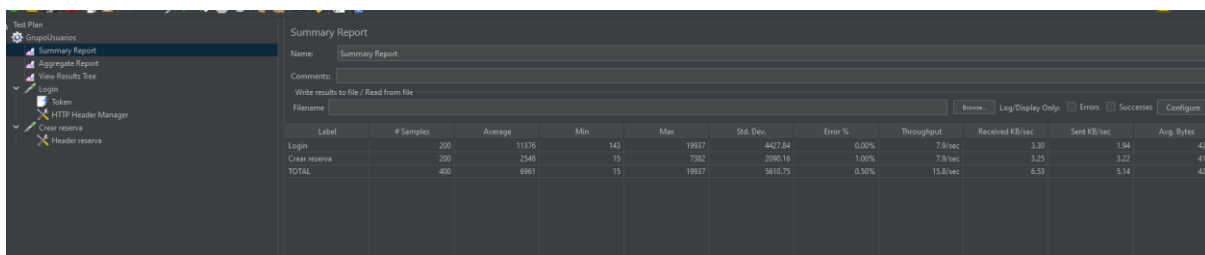
1. Sobre SimulaciónUsuarios100, se agregaron:
  - **Vista de árbol de resultados**
  - **Informe agregado**
  - **Informe resumen**



**Fig 40. Creación de Listeners – Grupo Usuario**

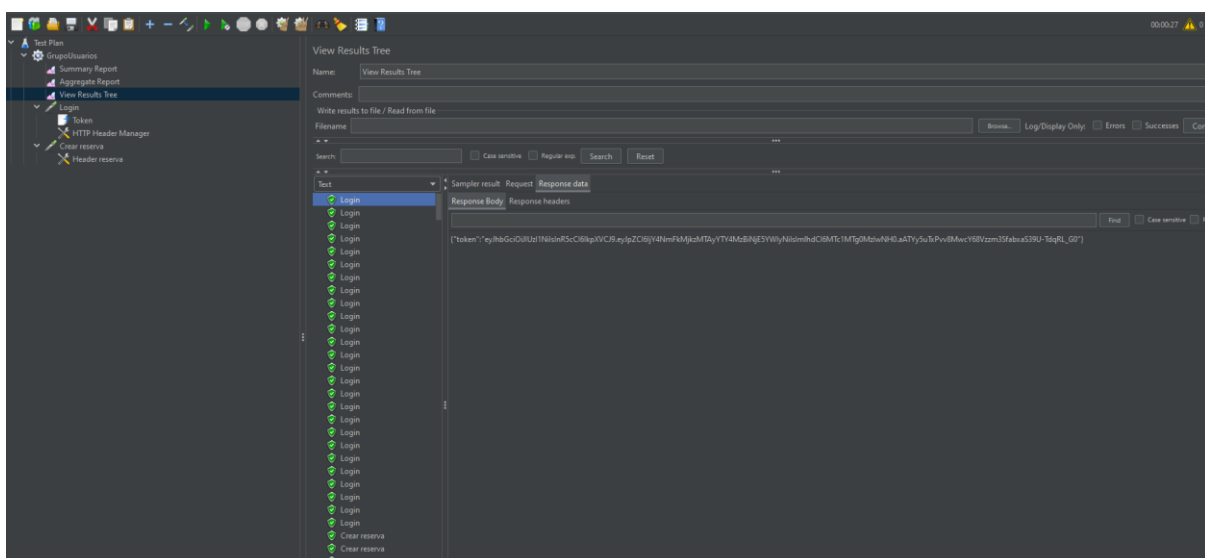
## Paso 6: Guardar y ejecutar la prueba

- Se guardó el plan como Test Plan.jmx.
- Se ejecutó la prueba con el botón.
- Se revisaron los resultados, tiempos de respuesta y errores.



| Label         | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Av. Bytes |
|---------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|-----------|
| Login         | 200       | 11376   | 143 | 19937 | 4427.84   | 0.00%   | 7.8/sec    | 3.30            | 1.94        | 420       |
| Crear reserva | 200       | 2546    | 15  | 7382  | 2090.16   | 1.00%   | 7.8/sec    | 3.25            | 3.22        | 419       |
| TOTAL         | 400       | 6961    | 15  | 19937 | 5610.75   | 0.50%   | 15.6/sec   | 6.53            | 5.14        | 421       |

**Fig 41. Ejecución de pruebas con 100 hilos – Summary Report**



**Fig 42. Ejecución de pruebas con 100 hilos – View Result Tree**

### Paso 7: Repetir prueba con 200 y 500 usuarios

- Se duplicó el grupo Grupo Usuarios.
- Se renombraron a 200 y 500.
- Se ajustó el número de hilos en cada grupo.
- Se volvió a ejecutar y se guardaron los resultados.

Número de hijos (usuarios)200:

Thread Group

Name: GrupoUsuarios

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 200

Ramp-up period (seconds): 10

Loop Count: ☐ Infinite ☒ 1

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

**Fig 43. Grupo Usuarios – 200 hilos**

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename:   ☐ Log/Display Only ☐ Errors ☐ Successes

| Label         | # Samples | Average | Min | Max  | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|---------------|-----------|---------|-----|------|-----------|---------|------------|-----------------|-------------|------------|
| Login         | 10        | 1903    | 433 | 2108 | 894.52    | 0.00%   | 2.5/sec    | 1.05            | 0.62        | 420.6      |
| Crear reserva | 7         | 1176    | 298 | 1770 | 555.31    | 0.00%   | 1.8/sec    | 0.71            | 0.75        | 399.8      |
| TOTAL         | 17        | 1863    | 298 | 2108 | 851.00    | 0.00%   | 4.3/sec    | 1.81            | 1.24        | 418.1      |

**Fig 44. Grupo Usuarios pruebas con 200 hilos - Listeners**

Pruebas\_Lab2

- admin
- config
- local
- reservas
  - reservas
  - usuarios

```

hora : "17:14b"
__V : 0

_id: ObjectId('686b016f695e961549c80e39')
usuario : ObjectId('686ad293182a6830b619ab26')
fecha : "06/07/2025"
sala : "C"
hora : "17:31"
__V : 0

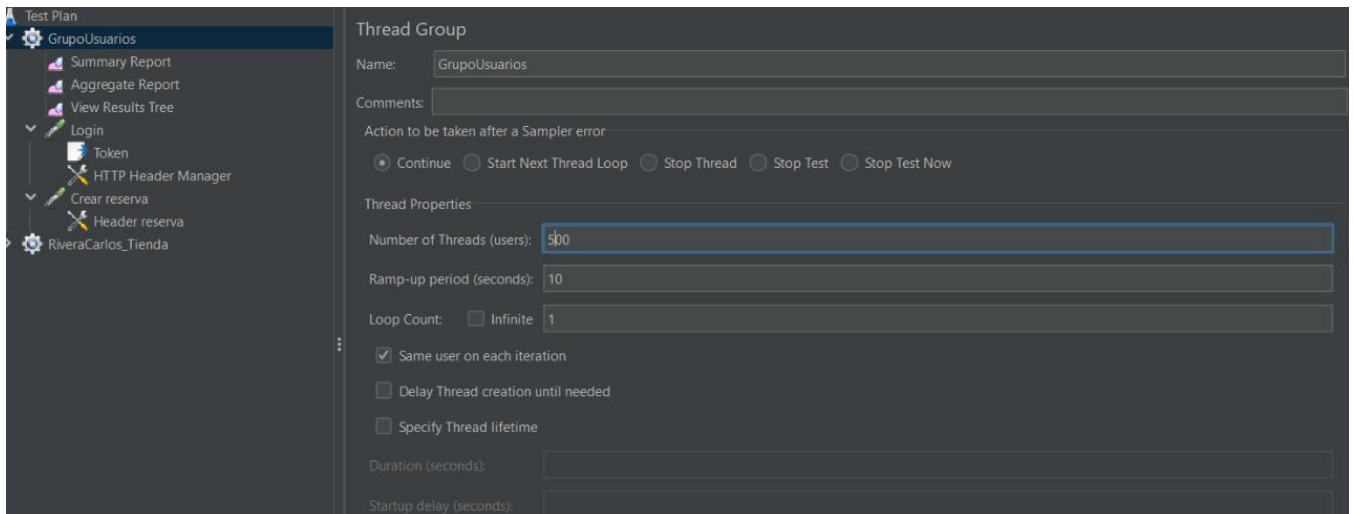
_id: ObjectId('686b016f695e961549c80e40')
usuario : ObjectId('686ad293182a6830b619ab26')
fecha : "06/07/2025"
sala : "C"
hora : "17:31"
__V : 0

_id: ObjectId('686b016f695e961549c80e42')
usuario : ObjectId('686ad293182a6830b619ab26')
fecha : "06/07/2025"
sala : "C"
hora : "17:31"
__V : 0

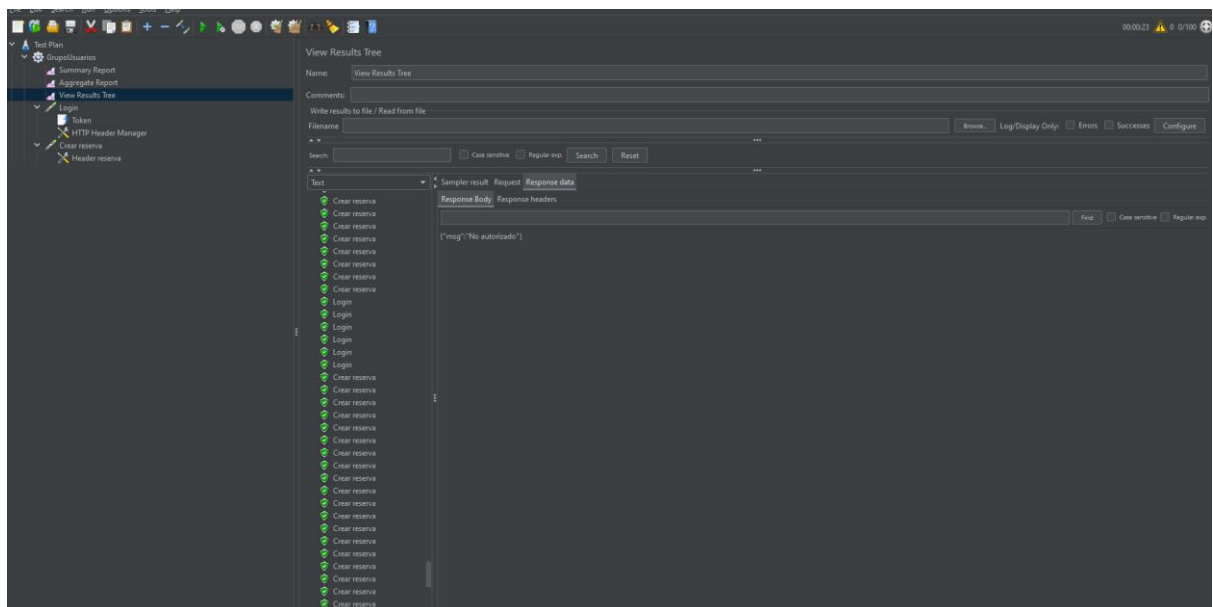
```

**Fig 45. Registro y Logins de las pruebas hechas en JMeter - MongoCompas**

Número de hijos (usuarios)200:



**Fig 46. Grupo Usuarios - prueba con 500 hilos**



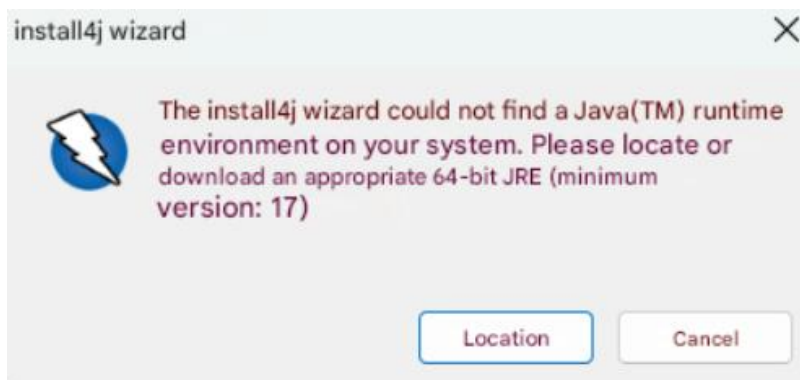
**Fig 47. Grupo Usuarios pruebas con 500 hilos - Listeners**

## SECCIÓN: Parte 4 - Pruebas de Seguridad con OWASP ZAP

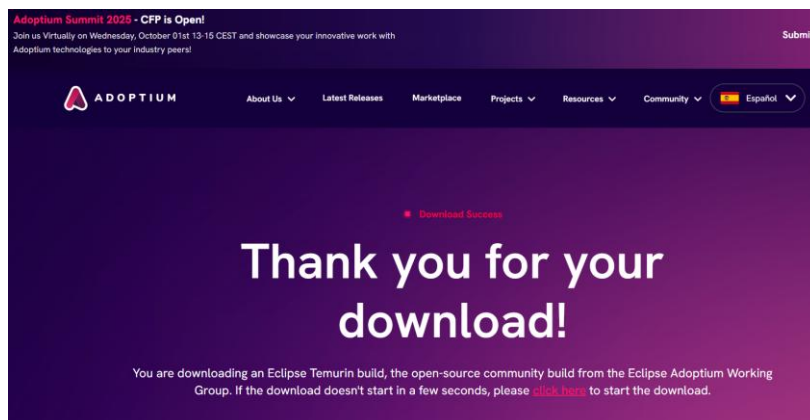
En esta parte se utilizan herramientas de análisis de seguridad automatizada mediante OWASP ZAP. Se evalúa si la API es vulnerable ante accesos indebidos, manipulación de tokens, rutas no protegidas y ataques comunes como inyección de SQL, escaneo tipo araña (Spidering), escaneo activo y fuzzing.

### Paso 1: Instalación y configuración de OWASP ZAP

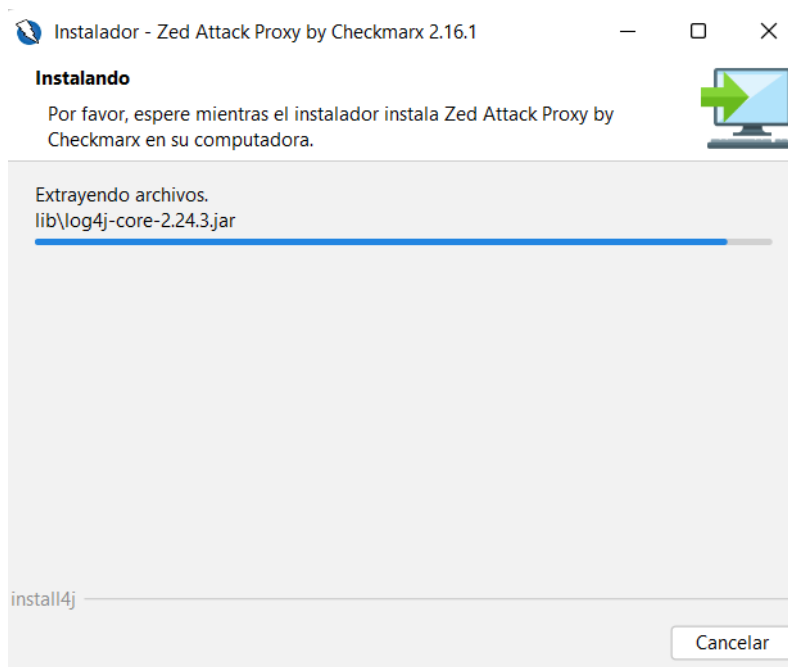
- Se descargó OWASP ZAP desde <https://www.zaproxy.org/>.



**Fig 48. Error por paquete JAVA (JDK) no instalado - ZAP**

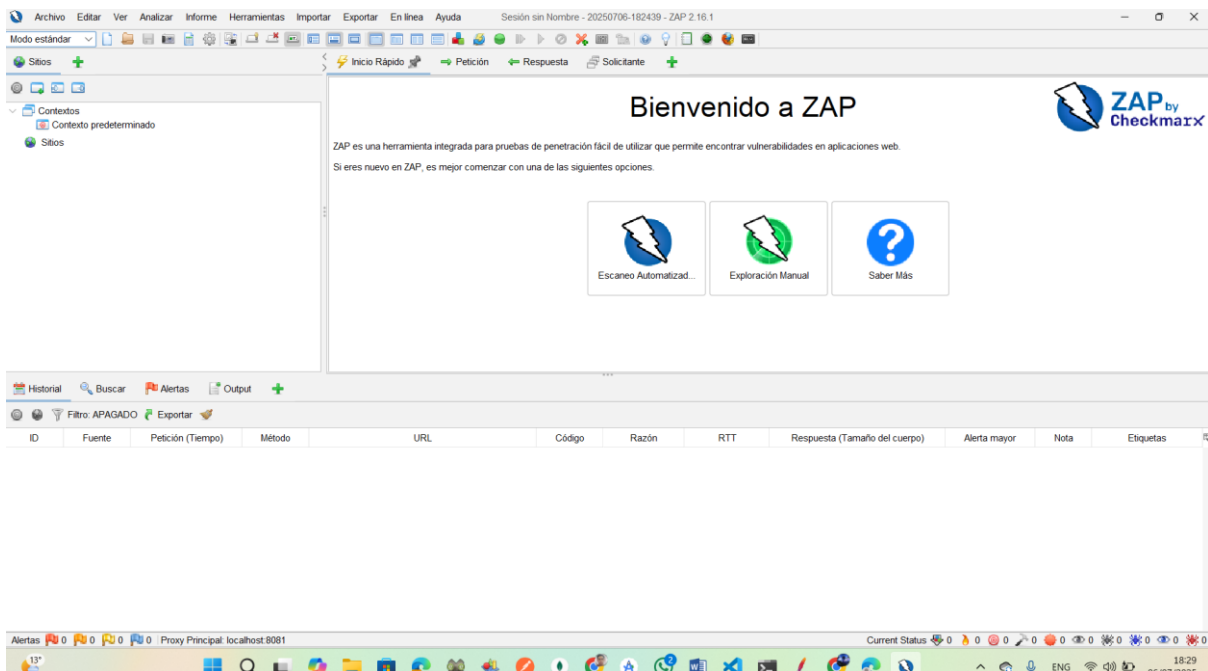


**Fig 49. Instalación de JAVA (JDK) - ZAP**



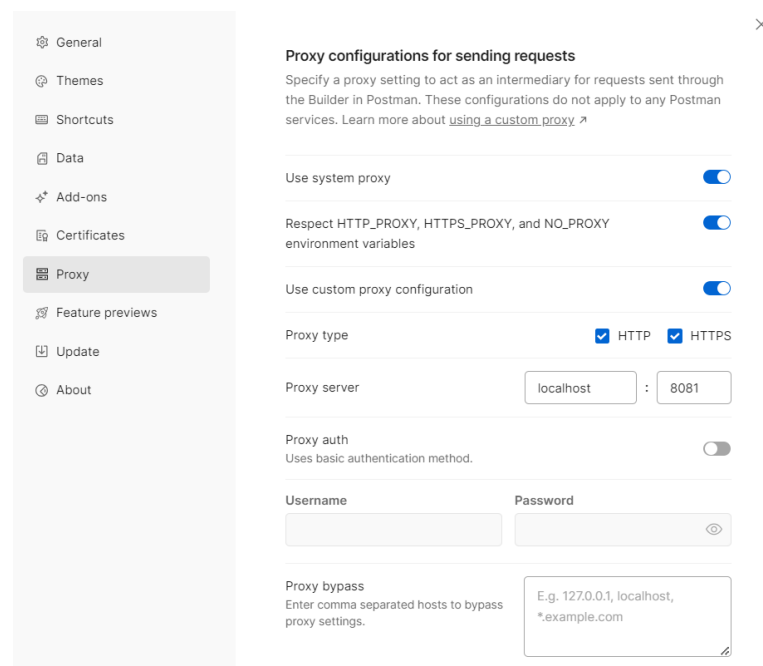
**Fig 50. Instalación de ZAP**

Se configuró el proxy:



**Fig 51. Página Principal de ZAP**

Host: localhost, Puerto: 8080



**Fig 52. Configuración de ruta POSTMAN para ZAP**

Se configuró Postman para que use el proxy de ZAP, permitiendo que todas las solicitudes pasen por la herramienta y puedan ser inspeccionadas.



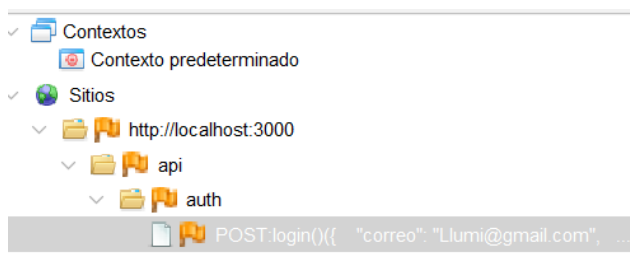


**Fig 53. Página Principal de ZAP conectada con POSTMAN**

## Paso 2: Navegación de API mientras ZAP está activo

Se ejecutaron peticiones a la API desde Postman (login, obtener reservas, crear y cancelar reserva).

ZAP comenzó a registrar automáticamente todas las rutas exploradas.



**Fig 53. Rutas de POSTMAN en ZAP**

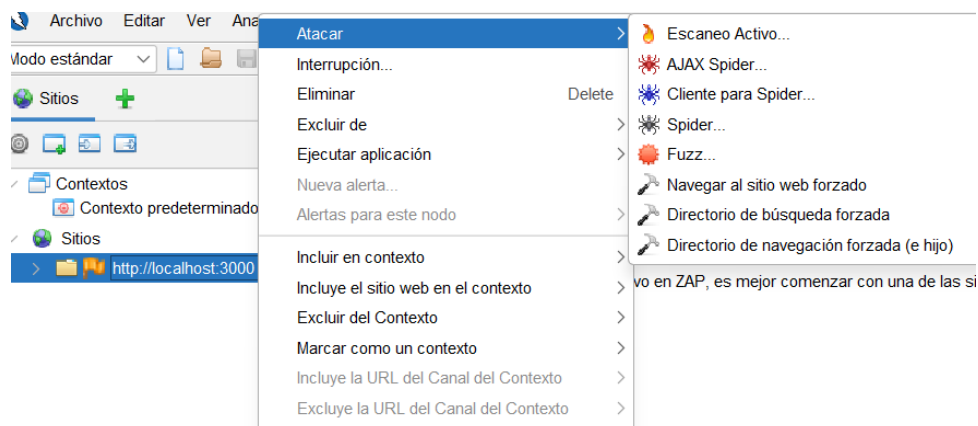
Panel izquierdo de ZAP con rutas descubiertas en el árbol de sitios (http://localhost:3000).

- ◇ Paso 3: Ataque 1 - Spidering (Escaneo tipo araña)
- ◇ Paso 4: Ataque 2 - Escaneo activo
- ◇ Paso 5: Fuzzing

## Empezar ataque 1: Spidering

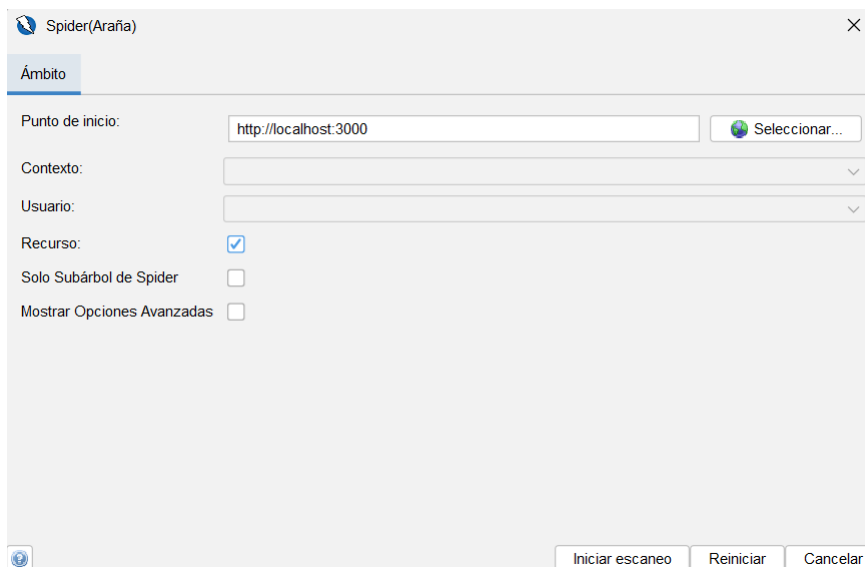
El Spidering es una técnica que permite descubrir rutas y recursos de una aplicación web automáticamente, simulando el comportamiento de un navegador que sigue enlaces. Esta función está integrada en OWASP ZAP y se usa como primer paso antes del escaneo activo.

- Se hizo clic derecho sobre http://localhost:3000 en el árbol de **Sitios** dentro del panel izquierdo de OWASP ZAP. Esta URL apareció automáticamente luego de enviar solicitudes a la API desde Postman con ZAP ejecutándose como proxy.
- En el menú desplegable, se seleccionó la opción **"Attack"** → **"Spider Site..."** para iniciar el escaneo tipo araña sobre esa URL.



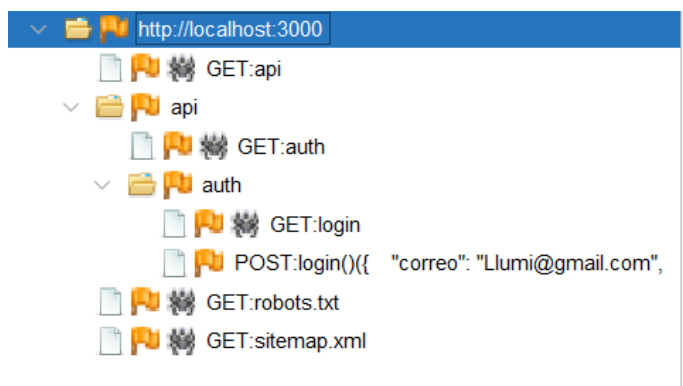
**Fig 54. Generar escaneo/ataque de araña**

- Se dejó la configuración por defecto, ya que se trataba de un entorno local controlado.
- Se hizo clic en **"Start Scan"** para comenzar el proceso.



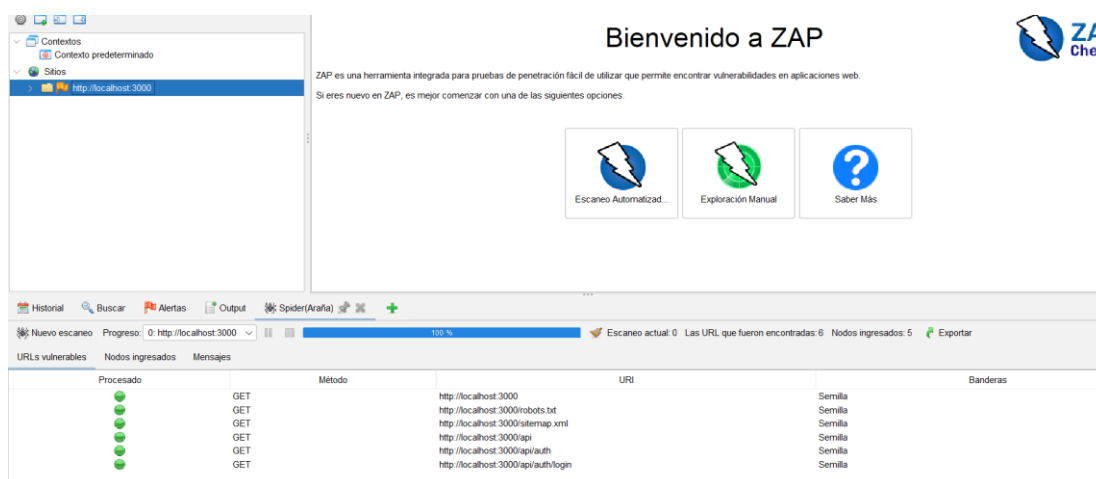
**Fig 55. Configurar la ruta del escaneo/ataque de araña**

1. Inicia el proceso.



**Fig 56. Inicio del escaneo/ataque de araña**

- En el panel izquierdo (**Sitios**), se empezaron a mostrar nuevas rutas detectadas, tales como:
  - /
  - /robots.txt
  - /sitemap.xml
  - /api
  - /api/auth
  - /api/auth/login
  - /api/reservas
- Estas rutas fueron marcadas como **semillas** del escaneo, y representan puntos de entrada útiles para futuras pruebas de seguridad más profundas.
- Se documentaron las rutas descubiertas y se validó que el spidering respetó los límites del dominio (localhost:3000), sin enviar solicitudes hacia otros dominios.



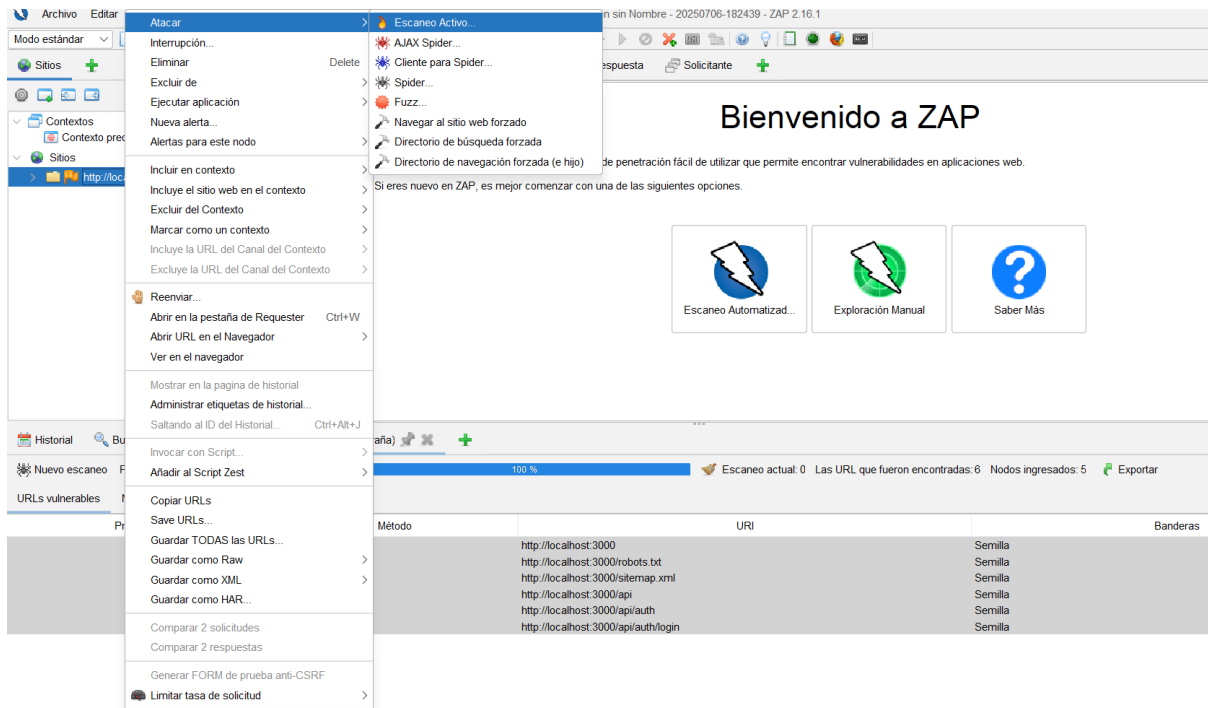
**Fig 57. Progreso del escaneo/ataque de araña**

Una vez iniciado el escaneo:

- ZAP comenzó a enviar múltiples solicitudes automáticas, recorriendo rutas, enlaces y parámetros disponibles en la API.
- En la parte inferior de la interfaz, se mostró una barra de progreso y un resumen del número de páginas descubiertas.

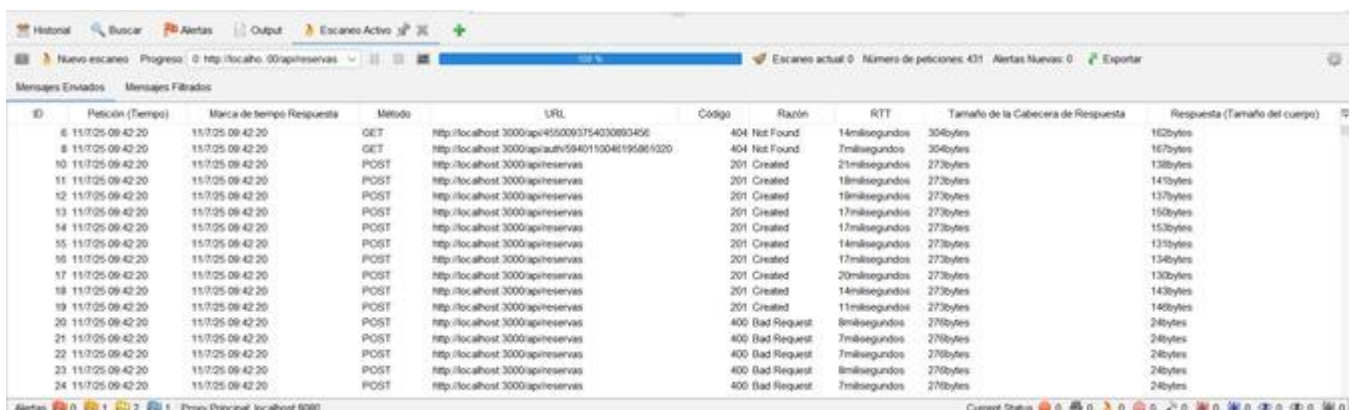
## Ataque 2: Escaneo Activo

El **escaneo activo** es una técnica ofensiva automatizada en OWASP ZAP que permite detectar vulnerabilidades en una aplicación web. A diferencia del spidering, este proceso envía peticiones especialmente diseñadas para provocar errores o revelar fallos de seguridad como inyecciones, rutas no protegidas, validaciones deficientes, entre otros.



**Fig 58. Generar escaneo activo**

- Se ejecutó un **escaneo activo** desde el sitio seleccionado (localhost:3000) con clic derecho > "Attack > Active Scan".
- ZAP comenzó a enviar múltiples peticiones automatizadas para probar:
  - Inyecciones SQL
  - Autenticación incorrecta
  - Parámetros inexistentes
  - Manipulación de URLs
  - Validación de errores y códigos de estado



The screenshot shows the 'Escaneo Activo' (Active Scan) results table in ZAP. The table has columns for ID, Petición (Tiempo), Marca de tiempo Respuesta, Método, URL, Código, Razón, RTT, Tamaño de la Cabecera de Respuesta, and Respuesta (Tamaño del cuerpo). The table contains 24 rows of data, showing various HTTP requests and responses, including 404 Not Found and 400 Bad Request status codes.

| ID | Petición (Tiempo) | Marca de tiempo Respuesta | Método | URL                                                | Código | Razón       | RTT            | Tamaño de la Cabecera de Respuesta | Respuesta (Tamaño del cuerpo) |
|----|-------------------|---------------------------|--------|----------------------------------------------------|--------|-------------|----------------|------------------------------------|-------------------------------|
| 6  | 11:07:25 09:42:20 | 11:07:25 09:42:20         | GET    | http://localhost:3000/api/4550093754030803456      | 404    | Not Found   | 14milisegundos | 304bytes                           | 162bytes                      |
| 8  | 11:07:25 09:42:20 | 11:07:25 09:42:20         | GET    | http://localhost:3000/api/auth/5940150048195965320 | 404    | Not Found   | 7milisegundos  | 304bytes                           | 167bytes                      |
| 10 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 21milisegundos | 273bytes                           | 138bytes                      |
| 11 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 18milisegundos | 273bytes                           | 141bytes                      |
| 12 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 18milisegundos | 273bytes                           | 137bytes                      |
| 13 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 17milisegundos | 273bytes                           | 150bytes                      |
| 14 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 17milisegundos | 273bytes                           | 153bytes                      |
| 15 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 14milisegundos | 273bytes                           | 131bytes                      |
| 16 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 17milisegundos | 273bytes                           | 134bytes                      |
| 17 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 20milisegundos | 273bytes                           | 130bytes                      |
| 18 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 14milisegundos | 273bytes                           | 143bytes                      |
| 19 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 201    | Created     | 11milisegundos | 273bytes                           | 146bytes                      |
| 20 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 400    | Bad Request | 8milisegundos  | 276bytes                           | 24bytes                       |
| 21 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 400    | Bad Request | 7milisegundos  | 276bytes                           | 24bytes                       |
| 22 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 400    | Bad Request | 7milisegundos  | 276bytes                           | 24bytes                       |
| 23 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 400    | Bad Request | 8milisegundos  | 276bytes                           | 24bytes                       |
| 24 | 11:07:25 09:42:20 | 11:07:25 09:42:20         | POST   | http://localhost:3000/api/reservas                 | 400    | Bad Request | 7milisegundos  | 276bytes                           | 24bytes                       |

**Fig 59. Análisis del escaneo activo**

### Resultados del Escaneo Activo:

- Se observaron múltiples códigos:
  - 404 Not Found: ZAP intentó rutas no válidas como /api/123456, /api/auth/9999, etc.
  - 401 Unauthorized: Peticiones hechas a rutas protegidas sin token válido.

- 200 OK: Algunas peticiones a `/api/auth/login` fueron exitosas, lo que indica que el escáner fue capaz de simular inicios de sesión.

**Ejemplo de ruta explorada:** POST `http://localhost:3000/api/auth/login` → 200 OK / 401 / 404

### Revisión de Alertas

- En la pestaña "Alertas" de ZAP se listaron posibles vulnerabilidades encontradas durante el escaneo.
- Cada alerta incluye:
  - Descripción del problema
  - Evidencia encontrada
  - Nivel de riesgo (Bajo, Medio, Alto)
  - Recomendaciones para mitigarlo

En este caso, ZAP no logró acceder a información sensible, pero generó alertas por rutas incorrectas, mal manejo de tokens o respuestas inconsistentes.

### Conclusión

El uso de **Spidering** y **Escaneo Activo** con ZAP permitió:

- Descubrir rutas de la API disponibles públicamente
- Evaluar la respuesta del backend ante entradas erróneas
- Detectar potenciales vulnerabilidades en autenticación y manejo de errores

Estas pruebas son fundamentales para garantizar la seguridad de aplicaciones web, especialmente en sistemas con rutas expuestas y lógica de autenticación sensible.

### Ataque 3: Fuzzing

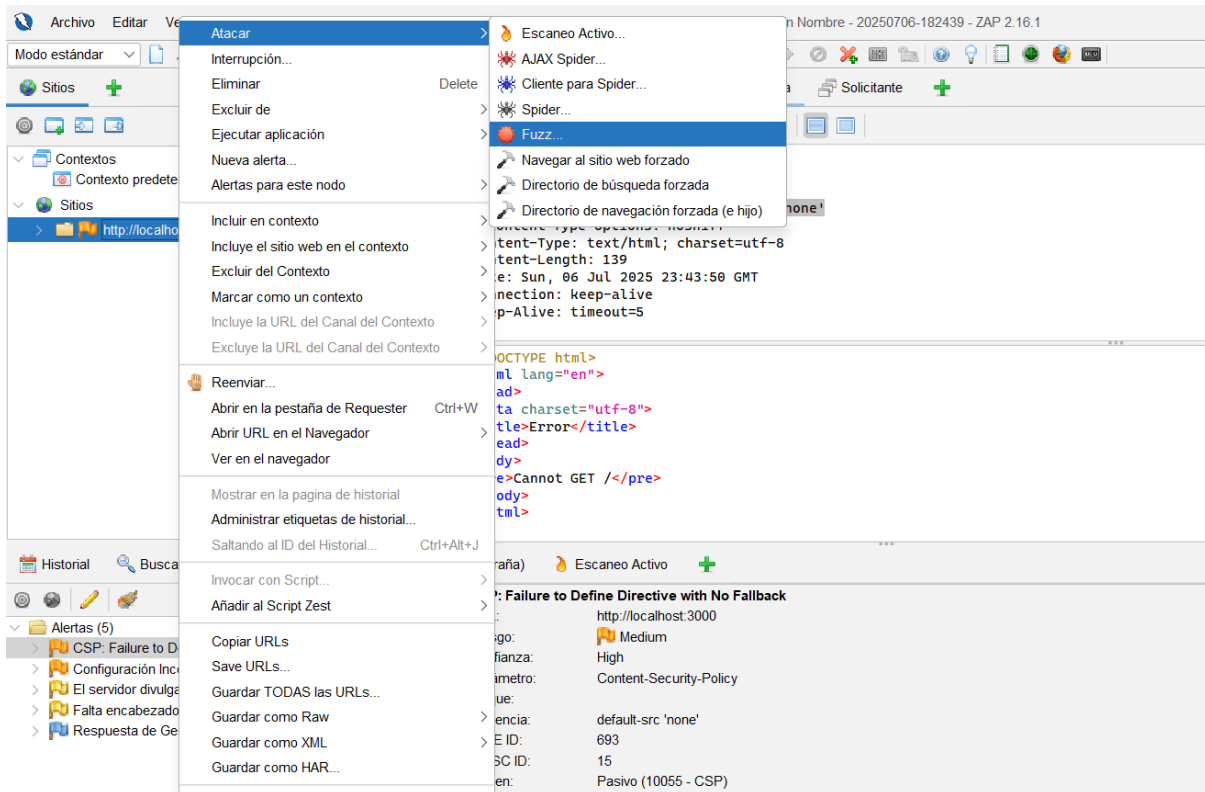
El Fuzzing es una técnica de pruebas donde ZAP envía grandes cantidades de datos maliciosos o aleatorios a entradas específicas de tu aplicación para ver si se comporta de forma inesperada o vulnerable.

#### ¿Qué busca el Fuzzing

- Inyecciones (SQL, XSS, comandos)
- Desbordamientos de búfer
- Errores de validación
- Respuestas inesperadas o fugas de información
- Comportamientos anómalos en parámetros, formularios, headers, etc.

### Paso 1: Selecciona un Request

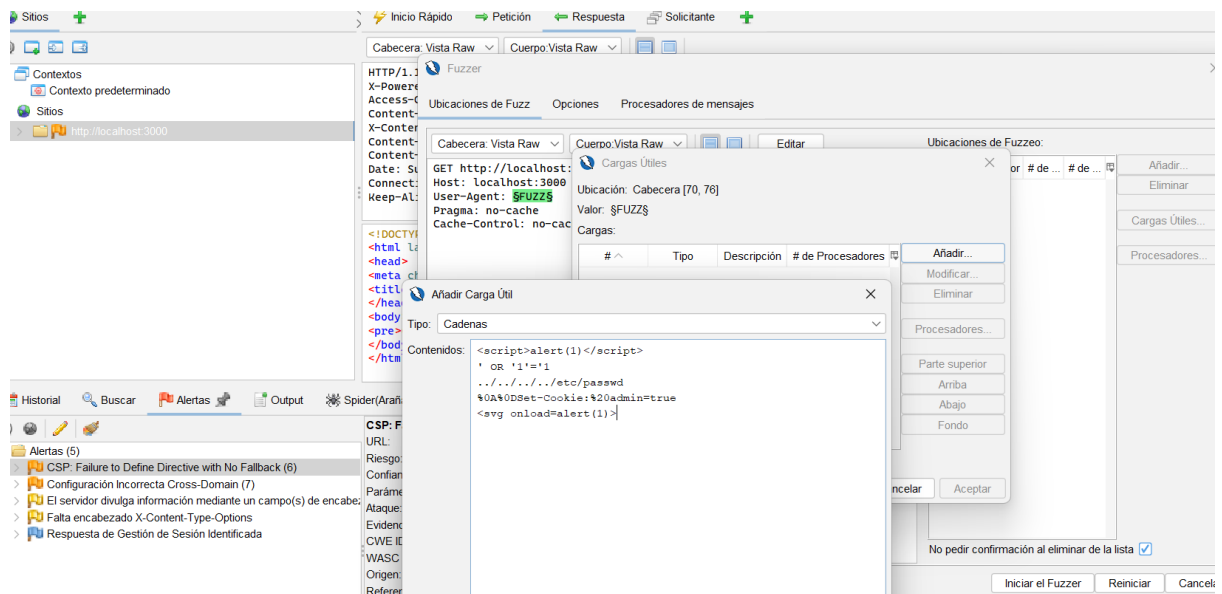
1. En el panel de la izquierda, en la sección "**Historial**" o "**Sites**".
2. Haz clic derecho sobre una petición (en este caso `http://localhost:3000/api/auth/login`).
3. Selecciona "**Attack**" → "**Fuzz...**"



**Fig 60. Generar el ataque Fuzz**

## Paso 2: Elegir el punto de inserción

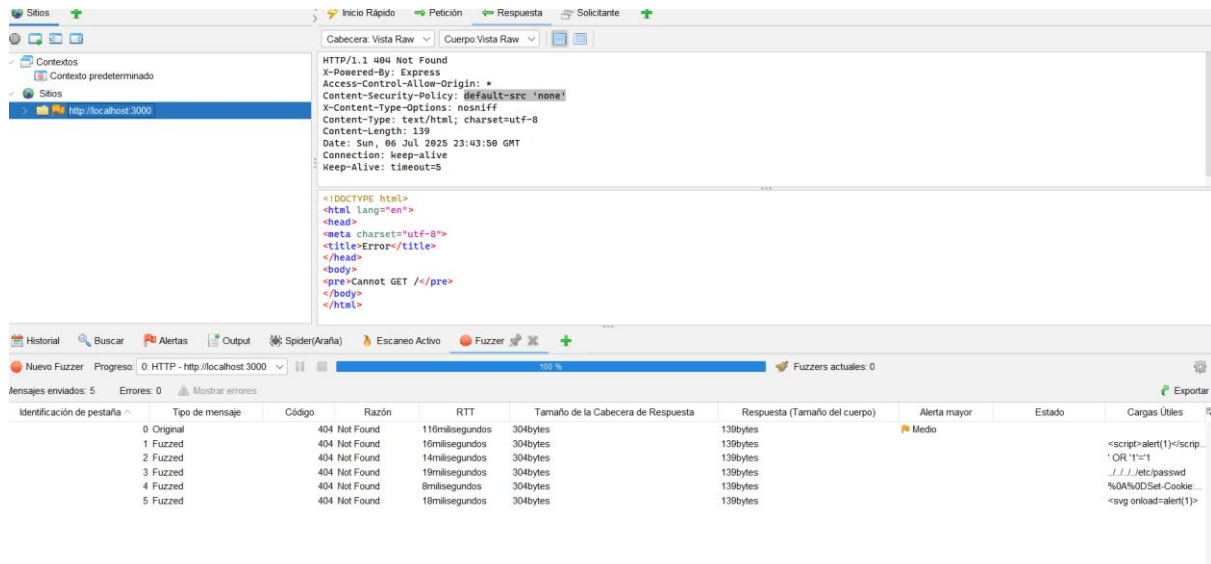
1. ZAP mostrará el cuerpo, headers y parámetros.
2. Seleccione el parámetro o campo donde se quería hacer.
3. Di clic en “Add” para añadir un payload.



**Fig 61. Selección de punto de inserción - ataque Fuzz**

### Paso 3: Ejecutar el Fuzzing

1. Di click en **Start Fuzzer**.
2. Y se aprecia cómo ZAP empieza a enviar los payloads al endpoint.



**Fig 62. Respuesta escaneo/ataque Fuzz**

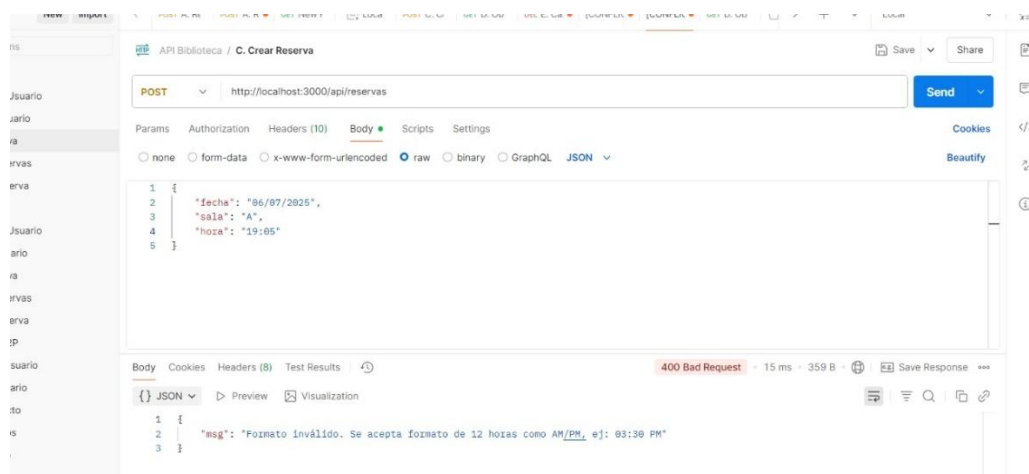
### 5. ACTIVIDAD /PREGUNTAS

Realizar pruebas de regresión teniendo en cuenta las siguientes solicitudes de modificación:

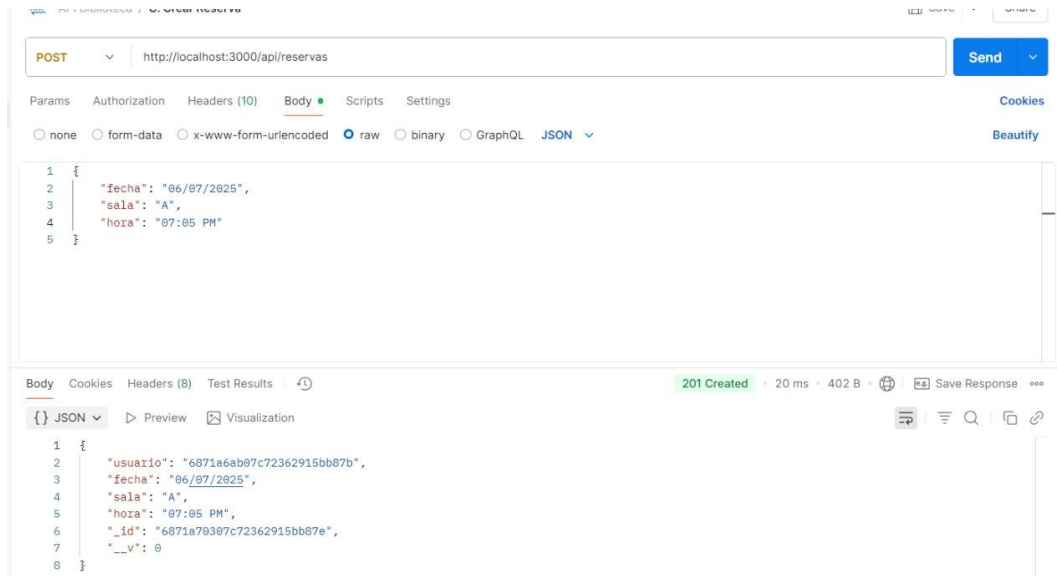
- a. Cambio solicitado 1: Por solicitud del administrador, la API debe aceptar ahora solo el formato de 12 horas con AM/PM (ej. "03:30 PM"). Las reservas enviadas en el formato anterior (24 horas) deben ser rechazadas con un código 400 y un mensaje explicativo.

```
// Validación de hora en formato 12h con AM/PM
const hora12Regex = /^(0[1-9]|1[0-2]):[0-5][0-9]\s?(AM|PM)$/i;
if (!hora12Regex.test(hora)) {
  return res.status(400).json({ msg: 'Formato inválido. Se acepta formato de 12 horas como AM/PM, ej: 03:30 PM' });
}
```

**Fig 63. Código cambiado – Parámetro hora**



**Fig 64. Prueba Creación con parámetro "hora" inválida**

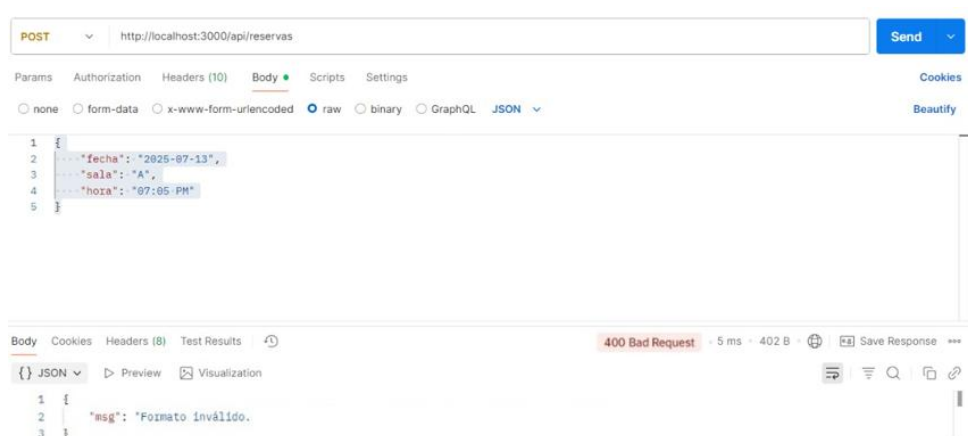


**Fig 65. Prueba Creación con parámetro “hora” válida**

- b. Cambio solicitado 2: Por solicitud de la administración, a partir de ahora no se permitirán reservas los días domingos. Cualquier intento de reservar un domingo debe responder con un error 400 y un mensaje: "No se permiten reservas los domingos".

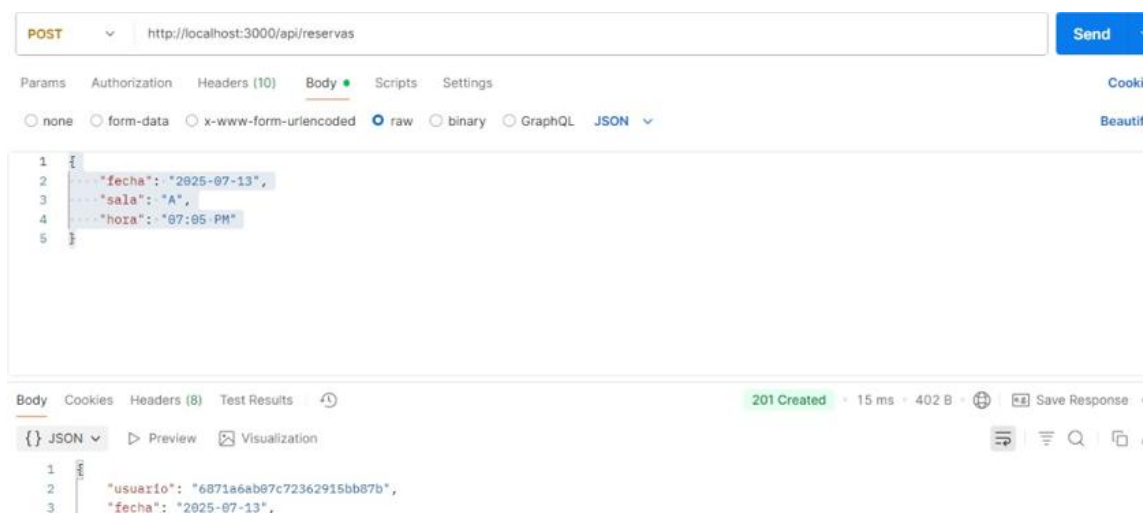


**Fig 66. Código cambiado – Parámetro domingo**



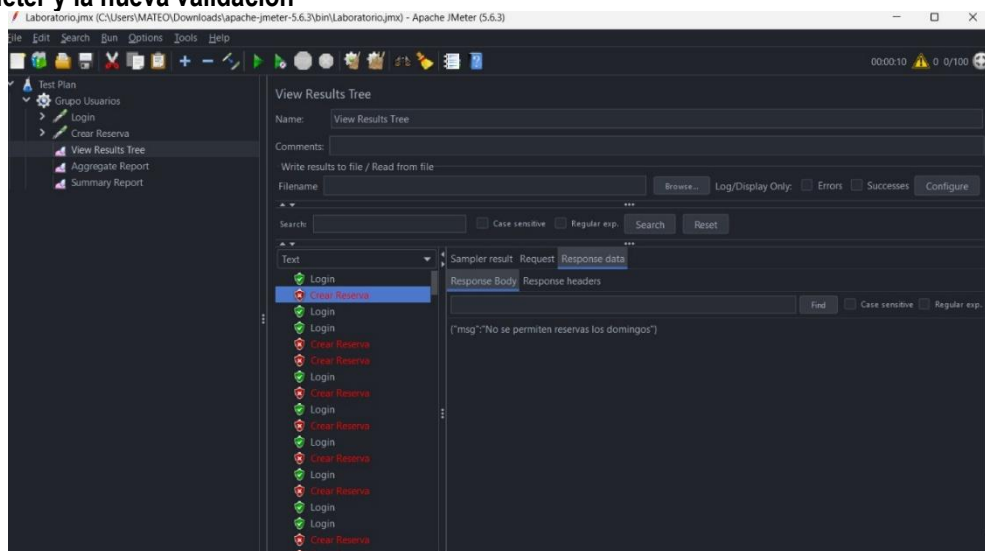
**Fig 67. Prueba Creación con parámetro “fecha” inválida**





**Fig 68. Prueba Creación con parámetro “fecha” válida**

### Prueba con JMeter y la nueva validación



**Fig 68. Prueba JMeter**

## 6. CONCLUSIONES:

La implementación estructurada de pruebas funcionales, de regresión, de carga y de seguridad permitió validar el correcto funcionamiento, la estabilidad y la robustez del sistema backend para la gestión de reservas. Las herramientas utilizadas —Postman, JMeter y OWASP ZAP— demostraron ser eficaces para detectar errores, verificar reglas de negocio, medir el rendimiento y evidenciar riesgos potenciales de seguridad.

La práctica fortaleció habilidades clave en la ejecución y documentación de pruebas profesionales, destacando la importancia de incorporar estos procesos en cada etapa del ciclo de desarrollo. Asimismo, se comprobó que el uso de herramientas automatizadas facilita la detección temprana de fallos y contribuye significativamente a la entrega de software más confiable y seguro.

**7. RECOMENDACIONES:**

- Es recomendable integrar herramientas de pruebas automatizadas como parte del ciclo de desarrollo continuo (CI/CD), de manera que se puedan detectar y corregir errores de forma temprana sin depender exclusivamente de pruebas manuales.
- Se sugiere reforzar las validaciones en la API, especialmente en los endpoints sensibles como autenticación y reservas, incorporando medidas adicionales de control de acceso, manejo de errores y validación de entradas para prevenir vulnerabilidades como inyecciones o manipulación de tokens.

**8. BIBLIOGRAFÍA:**

- OWASP Foundation. 2023. OWASP ZAP – Zed Attack Proxy, OWASP Project. Disponible en: [<https://www.zaproxy.org/>]. Fecha de consulta: 10 de julio de 2025.
- Apache Software Foundation. 2024. Apache JMeter – User Manual, Apache JMeter Project. Disponible en: [<https://jmeter.apache.org/>]. Fecha de consulta: 10 de julio de 2025.
- Postman Inc. 2024. Postman Learning Center – API Testing Documentation, Postman Docs. Disponible en: [<https://learning.postman.com/>]. Fecha de consulta: 10 de julio de 2025.
- Pressman, R. S., & Maxim, B. R. 2020. Ingeniería de Software: Un enfoque práctico, McGraw-Hill Education, Octava edición.
- Sommerville, I. 2016. Ingeniería del Software, Pearson Educación, Décima edición.
- Beizer, B. 1995. Software Testing Techniques, Van Nostrand Reinhold, Segunda edición.
- Myers, G. J., Sandler, C., & Badgett, T. 2011. The Art of Software Testing, John Wiley & Sons, Tercera edición.

**código Laboratorio:** [https://github.com/csriviera3/Examen\\_P3](https://github.com/csriviera3/Examen_P3)